



ACADEMIA DE STUDII ECONOMICE
Facultatea de Cibernetică, Statistică și Informatică Economică

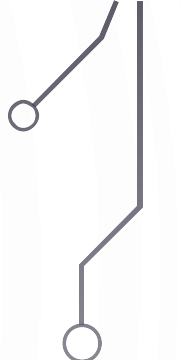
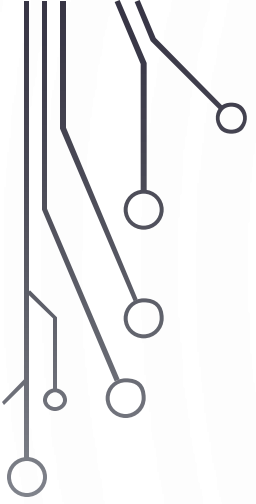
OBJECT ORIENTED PROGRAMMING IN C++

COURSE PRESENTATION

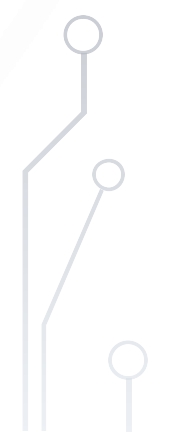
Prof. Cătălin BOJA
Economic Informatics and Cybernetics Department
catalin.boja@ie.ase.ro

A decorative graphic on the left side of the slide consisting of thin, light blue lines that resemble a circuit board or a network diagram. These lines are vertical and horizontal, with small circles at various points, suggesting nodes or connections. The lines are more dense on the left and become sparser towards the right.

**THE PURPOSE OF THIS DOCUMENT IS TO
STRUCTURE AND PRESENT BRIEFLY THE
CONCEPTS DISCUSSED DURING THE
COURSE. PREPARING FOR THE EXAM,
BASED SOLELY ON THIS MATERIAL
REPRESENTS A SUPERFICIAL APPROACH.**



The only way to learn to code in C++ (or other language) is to:

1. Understand the concepts
 2. Write a lot of code that Fails
 3. Fix the Code
 4. Repeat from step 2
- 
- 

COURSE OBJECTIVE & ASSESMENT

Objective

- To teach how to think and code in Object Oriented (OO) paradigm.....and to code in C++ (write source code, compile it, debug it and run it)

Course Assessment in 2025

- 50% seminar and lab activities
 - 10% of the final grade - laboratory live test
 - 15% of the final grade - laboratory live test
 - 5% of the final grade - seminar activities (homework, active participation, etc.)
 - 5% of the final grade - course activities (quizes, active participation, etc.)
 - 15% of the final grade - quiz in the last 2 weeks of the semester
- 50% final exam – practical exam that require the writing of a free compiler error C++ application that works

CONTENTS

- Review of necessary background
- Classes (definition, attributes, constructors, Destructor, methods, interface)
- Overloading operator methods
- Streams (standard and files)
- Inheritance (Frameworks – hierarchies of classes , Polymorphism, Virtual methods, Pointers to objects)
- Template classes
- STL – Standard Template Library

REFERENCES – FURTHER STUDY

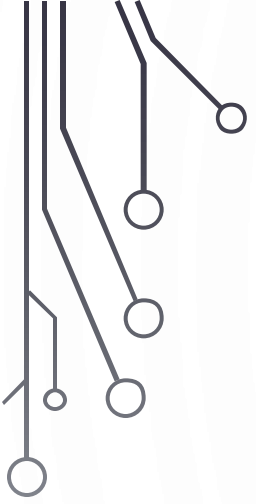
- www.acs.ase.ro/cpp
- Ion Smeureanu, Marian Dardala – “Programarea orientata obiect in limbajul C++”, Editura CISON, 2002
- Ion Smeureanu – “Programarea in limbajul C/C++”, Editura CISON, 2001
- The standard: Bjarne Stroustrup – The Creator of C++, “The C++ Programming Language”-3rd Edition, Addison-Wesley, <http://www.research.att.com/~bs/3rd.html>
- <https://en.cppreference.com/w/>
- Any tutorial found on Internet

GENERAL RULES

- You need to write a lot of C++ programs – test all the concepts you read, see and hear
- Getting errors during development is the recommended path – fix them and understand why you got them
- Don't write only perfect programs
- All assignments and tests are individual
- Copying at any activity (home assignments, tests, project, exam) may get you expelled but it will surely cancel all your laboratory points

WHY OBJECT ORIENTED

- 3 magic words: **Encapsulation, Inheritance, Polymorphism**
- Concepts are present in all Object Oriented Languages (.NET platform - C#, Java, Kotlin, C++, Python, Objective-C) or Object Based Languages (JavaScript)
- For a long time, one of the top 5 used programming languages
<https://www.tiobe.com/tiobe-index/>

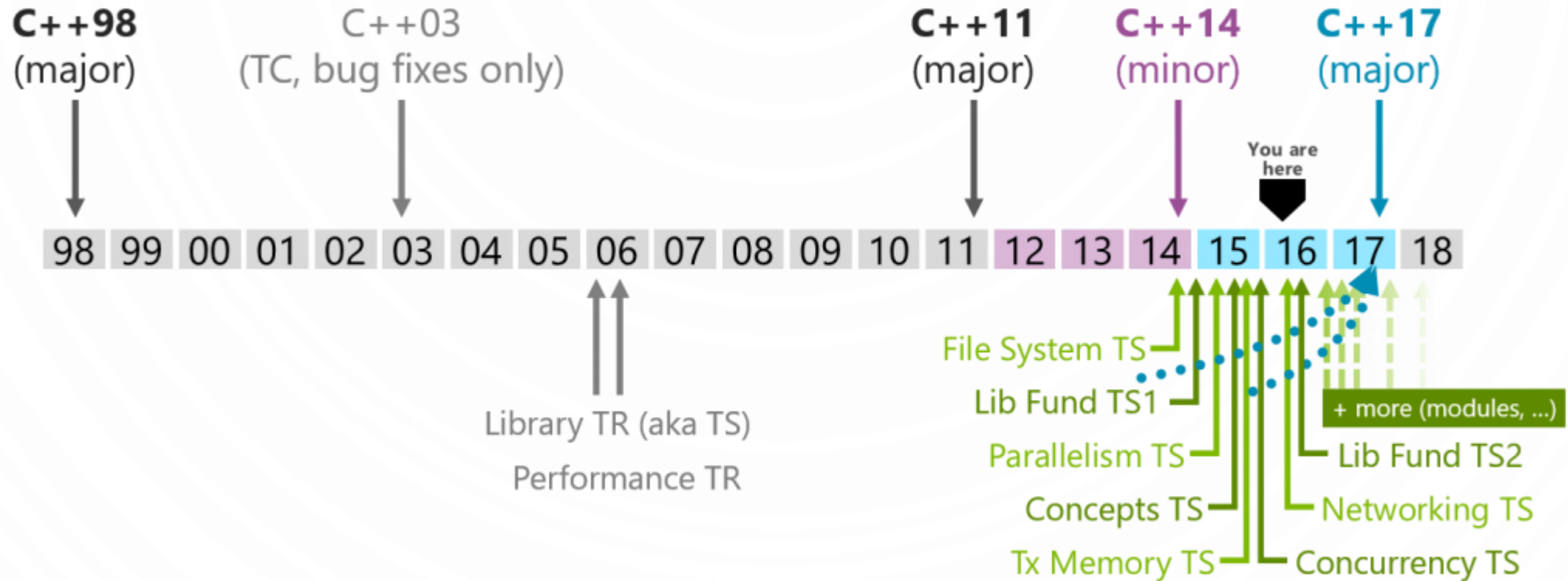


WHY OBJECT ORIENTED

Because is all about abstraction

Taking a real-life story and translate it into code

C++ EVOLUTION



Source <https://www.quora.com/Computer-Programming-As-a-web-developer-why-should-I-learn-C-or-C++>

WHY C++

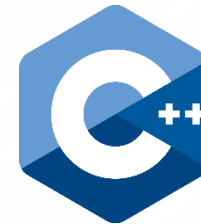
- It's all about Object Oriented Programming
 - You will be able to design software solutions for real life problems – abstraction
 - Used for Web development (Java Spring, NodJS, JavaScript, Python, .Net Core and any language that implements a MVC based framework), Embedded devices, Mobile Devices and Client Side programming
 - You will learn faster other OO languages (like Java and C#)
- The C++ part is the syntax

THE CONTENT OF THIS COURSE

- Objects
- Classes
- Inheritance
- Encapsulation
- Frameworks of classes
- Abstraction



- C++ syntax



WHY C++

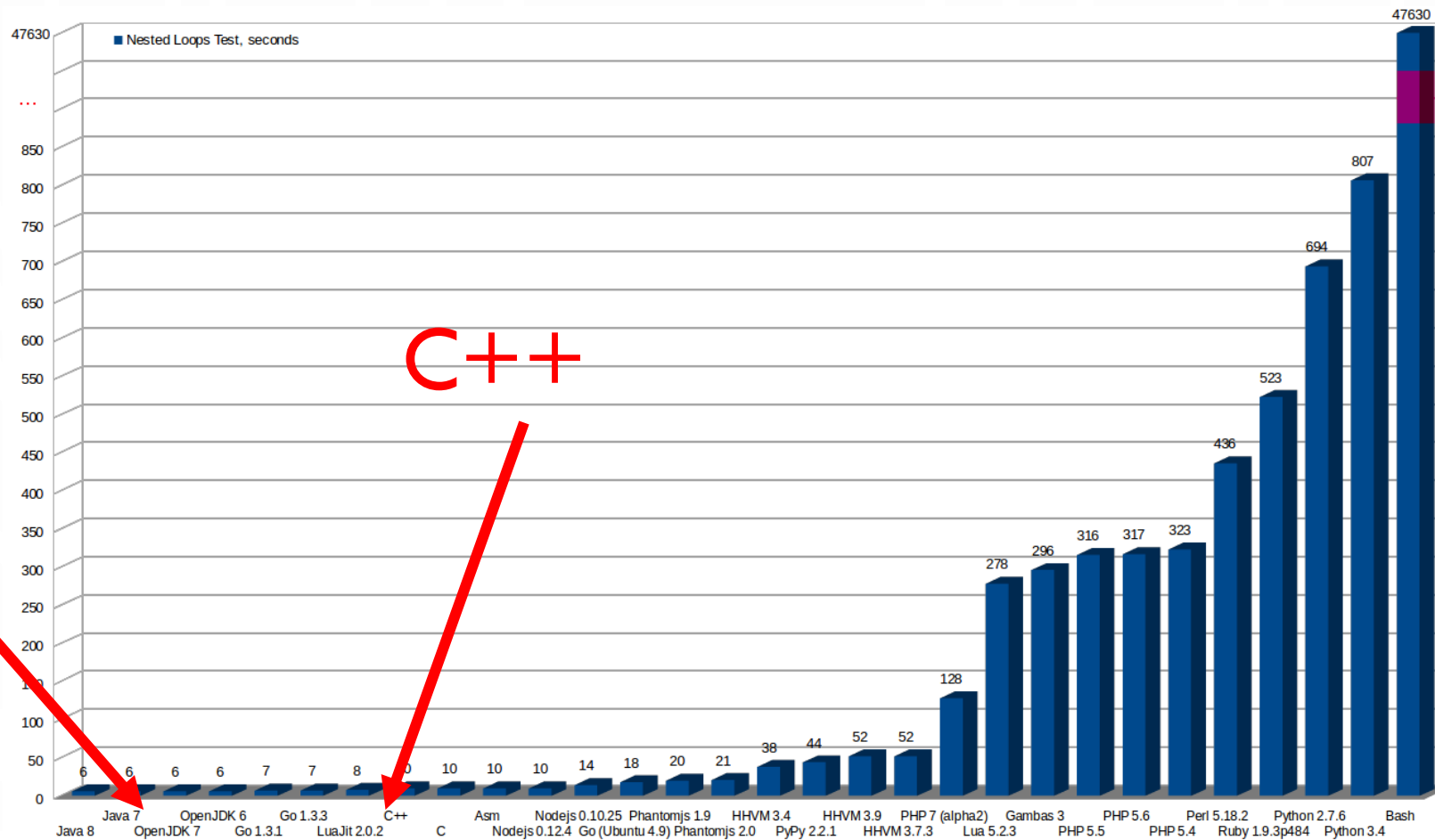
- Very fast
- Provides specific implementation for a lot of hardware
- No need for a virtual machine or an interpreter
- For many years in top 5 used programming languages in the TIOBE index (3rd in October 2017 and 4th now in October 2018) -

<https://www.tiobe.com/tiobe-index/>

WHY C++

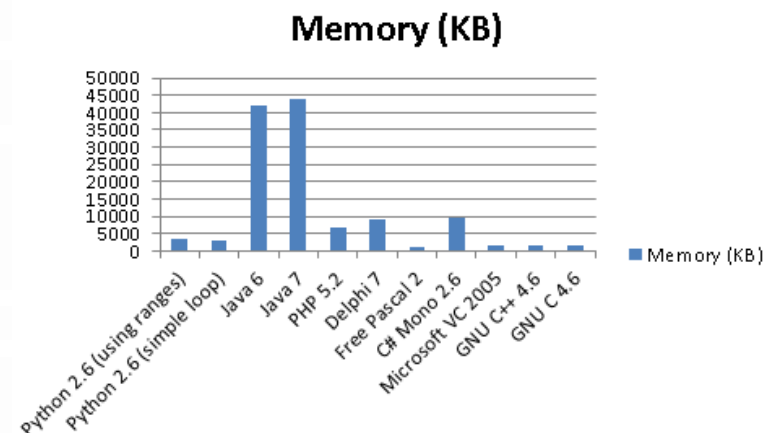
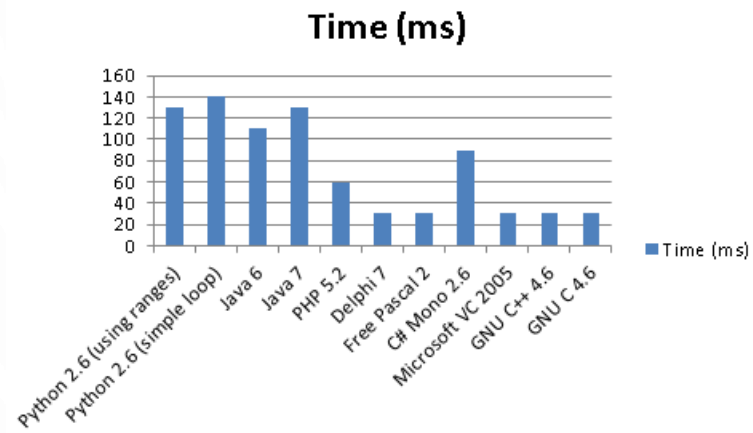
Java

C++



<http://blog.carlesmateo.com/2014/10/13/performance-of-several-languages/>

WHY C++



<https://helloacm.com/a-quick-performance-comparison-on-languages-at-codeforces/>

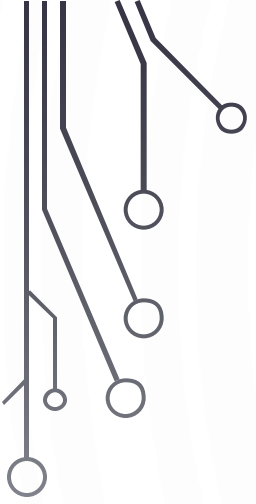
WHY C++

 C++	 JavaScript
 Java/C#	 PHP(Without MySQL)
 Ruby	 Pascal
 Perl	 Lisp
 Visual Basic	 Haskell
 Python	 C

@左耳朵耗子
weibo.com/hasei

LEARNING TO CODE

- **Learning how to code is based on numerous failures**
 - you need to try to solve different problems using C++ by yourself (after you think you got the concepts) and this will lead to a lot of failures
 - solving all the problems (even if takes time.....sometimes too much) will help you learn
- **You need to**
 - Accept that there is always room for improvement (nobody is perfect)
 - Be persistent in achieving your goals
 - Use your imagination to find solutions
- **Don't cheat**



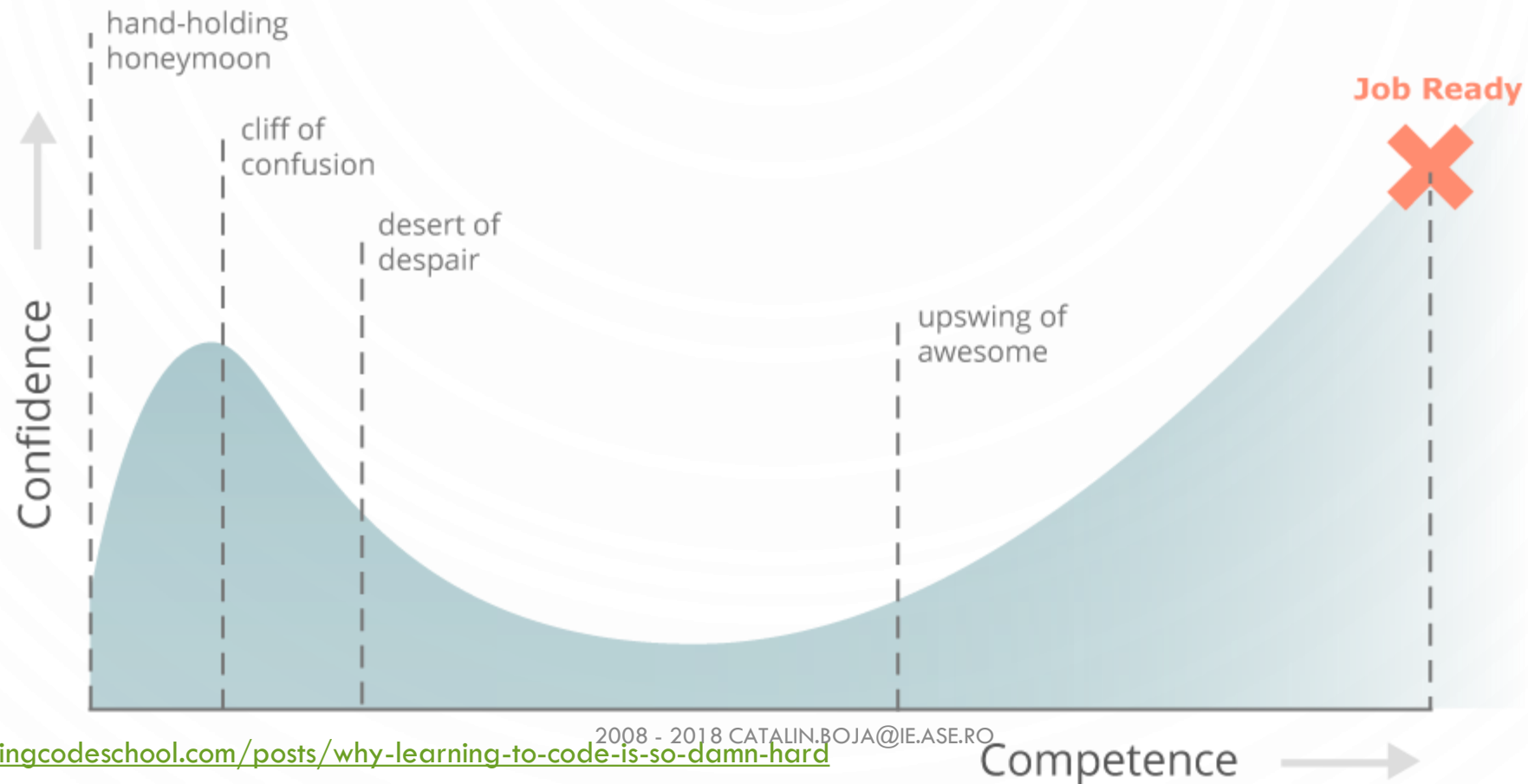
HOW TO LEARN

“We learn wisdom from failure much more than from success. We often discover what will do, by finding out what will not do; and probably he who never made a mistake never made a discovery”

— **Samuel Smiles, The Lives Of George And Robert Stephenson**

LEARNING TO CODE....IN C++ AND OTHERS

Coding Confidence vs Competence



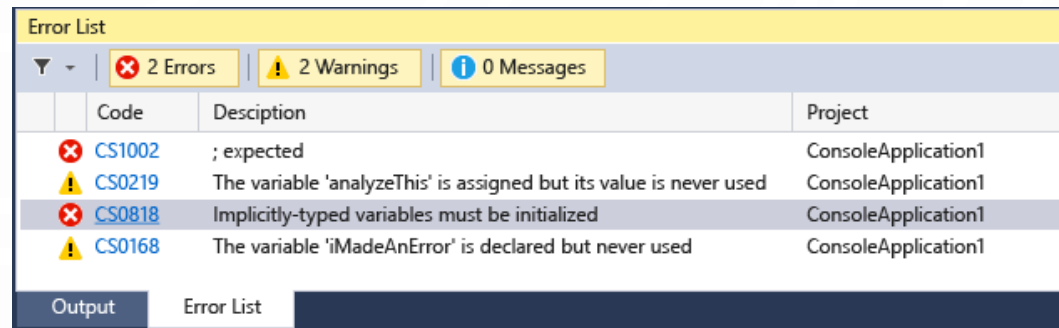
MAIN REASONS FOR FAILING

- Not enough C ++ programs are implemented in Visual Studio
- Not enough C ++ programs are implemented in Visual Studio
- Not enough C ++ programs are implemented in Visual Studio
- Not enough C ++ programs are implemented in Visual Studio
- Not enough C ++ programs are implemented in Visual Studio
- Not enough C ++ programs are implemented in Visual Studio
- Not enough C ++ programs are implemented in Visual Studio
- Not enough C ++ programs are implemented in Visual Studio
- Not enough C ++ programs are implemented in Visual Studio
- Not enough C ++ programs are implemented in Visual Studio
- I do not understand the concepts

MAIN REASONS FOR FAILING

Compiler errors

- Generated by code editing errors
- > 99% because syntax is not known
- Clearly highlighted by the compiler in the Errors (VS) windowand Warnings one
- NOT related to program complexity, algorithm used, etc



The screenshot shows the 'Error List' window in Visual Studio. At the top, there are tabs for '2 Errors', '2 Warnings', and '0 Messages'. Below this is a table with columns for 'Code', 'Description', and 'Project'.

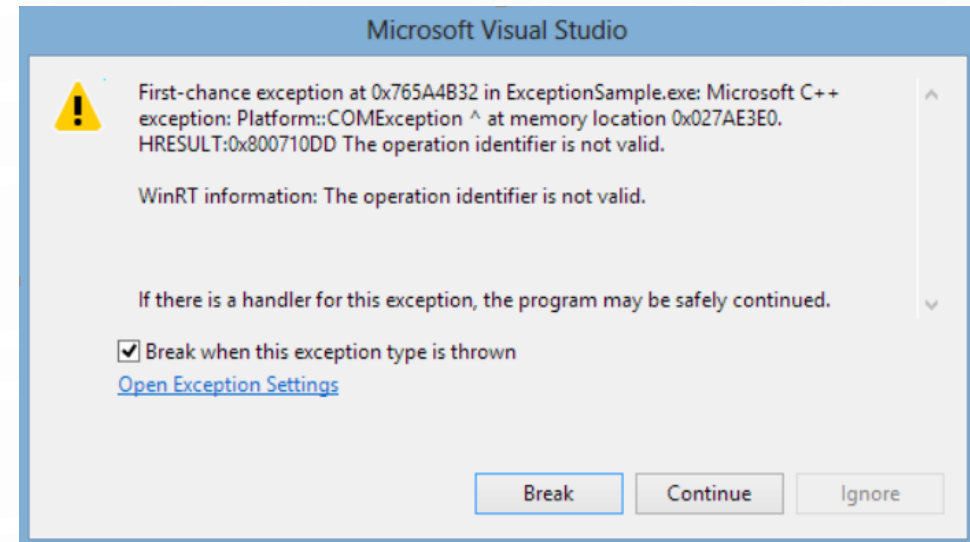
	Code	Description	Project
✖	CS1002	; expected	ConsoleApplication1
⚠	CS0219	The variable 'analyzeThis' is assigned but its value is never used	ConsoleApplication1
✖	CS0818	Implicitly-typed variables must be initialized	ConsoleApplication1
⚠	CS0168	The variable 'iMadeAnError' is declared but never used	ConsoleApplication1

At the bottom of the window, there are tabs for 'Output' and 'Error List', with 'Error List' being the active tab.

MAIN REASONS FOR FAILING

Run-time errors

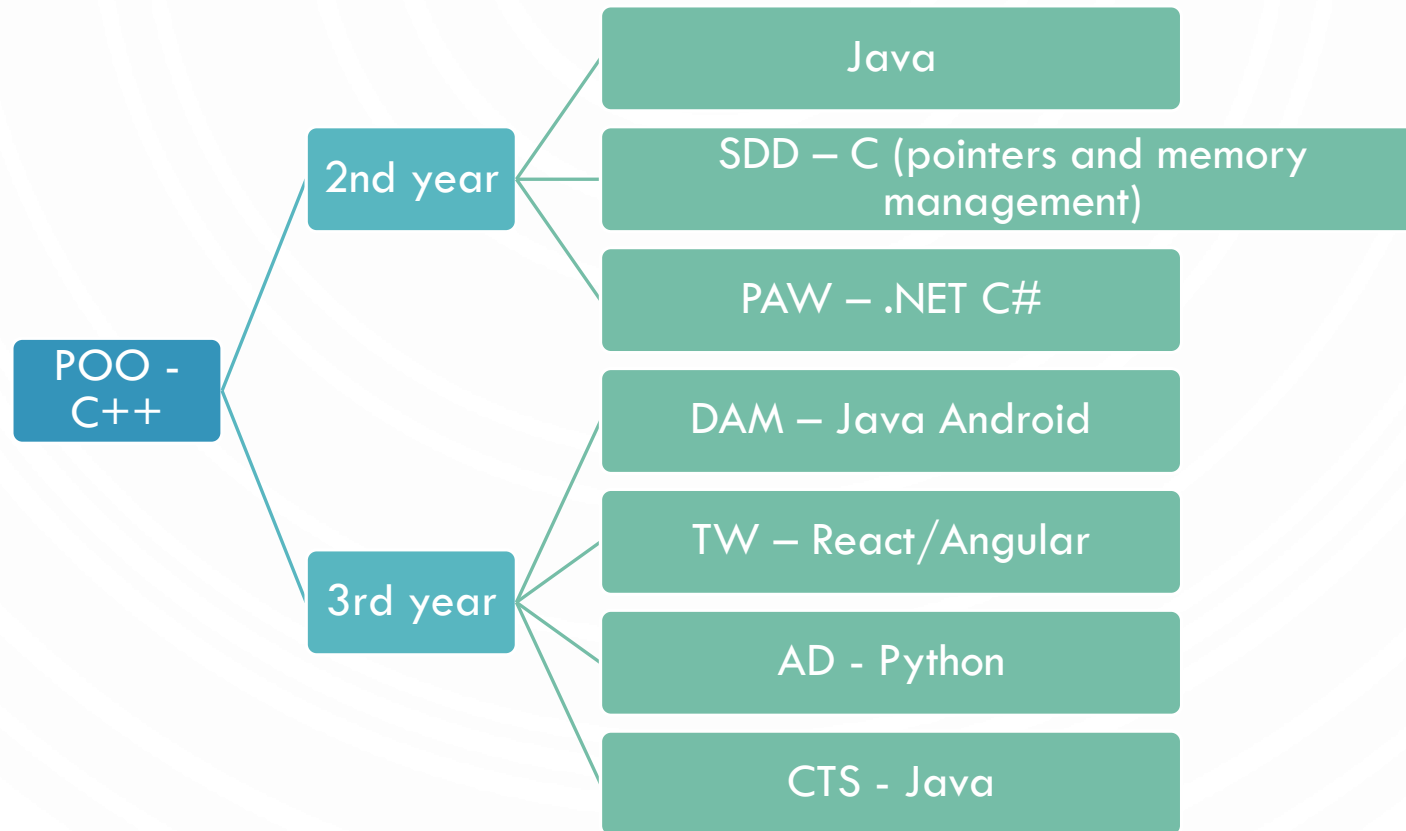
- Generated by mistakes in the logical implementation of the program
- > 99% because faulty or bad memory management (pointers, heap, stack, allocation, deallocation, etc.)
- Hard to identify and correctyou need to use the debugger
- They are related to the complexity of the program, the algorithm used, etc



RECIPE FOR SUCCESS

1. After each course and seminar, check if you understood the concepts (otherwise read and ask colleagues and teachers)
2. Write as many C ++ programs as possible (if all are ok from the first attempt this is not okay) ... but do not copy them from the course and seminar examples (define your own examples)
3. If all the initial attempts have compilation and / or execution errors you are on the right track
4. If you are in step 2, do not give up fix them by reading the course support and looking for resources on the Internet (not everything you find on stackoverflow is right)
5. If you DO NOT succeed in 2-3 hours to correct a problem ask for help (colleagues and teachers)
6. DO NOT give up writing as many programs as possible ... you need to code 2-3 hours a week

WHAT FOLLOWS



COURSE 1 – WHAT TO REMEMBER

1. Data types (possible values and size)
2. Hexadecimal and binary representation
3. Functions/Methods/Subprograms (definition and calling)
4. At least one sorting algorithm
5. At least one searching algorithm
6. Using VS 2019 (editing/compiling/running and debugging)

COURSE 1 - TOOLS



Microsoft Visual Studio 2017 or 2019

<https://www.visualstudio.com/downloads/>



Eclipse CDT

<https://eclipse.org/cdt/>

COURSE 1 – RECAP SOME C CONCEPTS

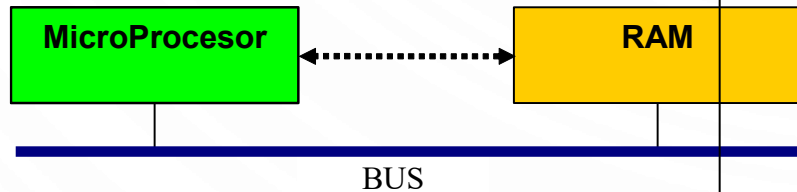
- Pointers (pointers to values, static and dynamic arrays, strings or arrays of chars)
- Function pointers
- References
- Functions/Methods (parameters transfers)
- Preprocessing

POINTERS

C/C++ Source

```
#include<stdio.h>

void main()
{
    char a = 7, b = 9;
    short int c;
    c = a+b;
}
```



ASM

```
.model small
.stack 16
.data

a db 7
b db 9
c dw ?

.code
start:

    mov AX, @data
    mov DS, AX

    mov AL,a
    add AL,b
    mov c,AX

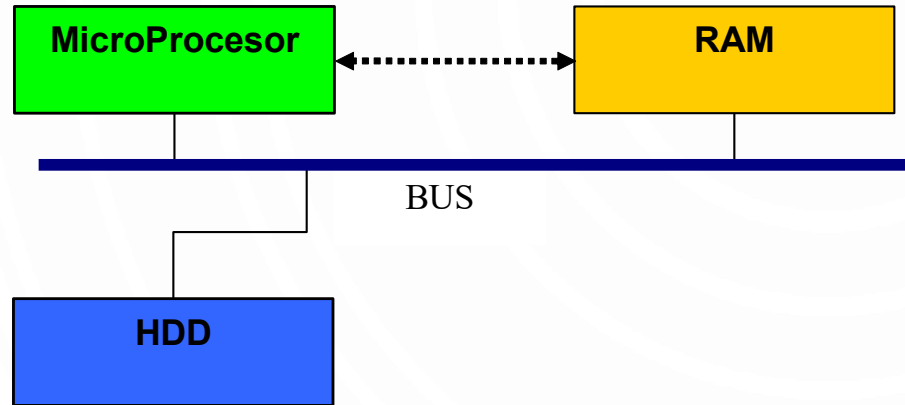
    mov AX, 4C00h
    int 21h

end start
```

Machine code

```
B8 02 00 8E D8
A0 00 00 02 06
01 00 A3 02 00
B8 00 4C CD 21
00 00 00.....00 00
07 09
```

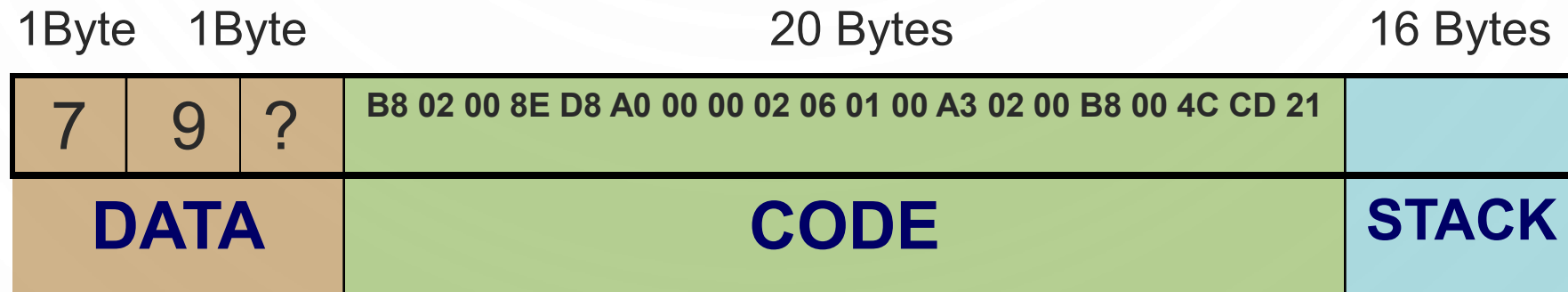
POINTERS



C/C++ Source

```
#include<stdio.h>

void main()
{
    char a = 7, b = 9;
    short int c;
    c = a+b;
}
```



POINTERS

- Numerical variables used to store values that represent memory addresses;
- Their size is given by the processor architecture (on 32 bits -> 4 bytes)
- definition:

```
data_type * pointer_name;
```

- init:

```
pointer_name = & variable_name;
```

- usage:

```
variable_name = * pointer_name ;
```

POINTERS

- **Example:**

```
i n t * p i ; // pointer to int
```

```
c h a r ** p p c ; // pointer to pointer of char
```

```
i n t * a p [10] ; // array of 10 pointers to int
```

- Zero value for a pointer is a **null** value. It is defined using the next macro

```
#define NULL 0
```

- or by the next constant value

```
const int NULL = 0;
```

POINTERS

Pointers arithmetic:

- For a T type pointer T^* **pointer**, the **operatorii --/++ operators** will move the current address back/forward with **sizeof(T)** octeti;
- For a T type pointer T^* **pt**, the next expression **pt + k** or **pt - k** is equivalent with moving the current address with **k * sizeof(T)** bytes;
- Adding 2 pointers has no logic and is not accepted (use 2 integers instead);
- Substraction is allowed

CONSTANT POINTERS

- Examples:

<code>int * const p;</code>	<code>// constant pointer to <i>int</i></code>
<code>int const * pint;</code>	<code>// pointer to constant <i>int</i></code>
<code>const int * pint2;</code>	<code>// pointer to constant <i>int</i></code>
<code>const int * const pint2;</code>	<code>// constant pointer to <i>int</i> constant</code>

- Usage:

```
char* strcpy(char* p, const char* q);
```

- - prevents changing q pointer

POINTERS

Allocating dynamically memory (in HEAP):

- `new` or `new [ ]`;
- Always in Heap

Deallocating HEAP memory space:

- `delete` or `delete[ ]`;
- Only for HEAP

C++ REFERENCE

- Represents a constant pointer that is used only to access values located at the referenced memory space
- Used to define the input arguments of a method (simpler syntax than *)

```
int vb = 10 ;  
int & refvb = vb ;  
int x = refvb ;  
refvb = 20 ;  
int & ref;  
refvb ++;  
int * pvb = & refvb;
```

```
// r and i now refer to the same int  
// x = 10  
// vb = 20  
//ERROR – compiler error  
// vb = 21  
// pvb is initialized with vb address
```

POINTER – TO METHODS

- definition:

```
return_type (* pointer_name) (arguments list);
```

- init:

```
pointer_name = function_name;
```

- usage:

```
pointer_name (arguments list);
```

POINTER – TO METHODS

- `float (*f p)(int *);` // pointer to a method that returns a float value and receives a pointer to int
- `int * f(char *);` // **NOT a pointer** – just a method called f that returns a pointer to int
- `int * (*f p[5])(char *);` // array of 5 pointer

PREPROCESSING

- Happens before compilation
- A language for the compiler
- Using symbols defined with #
- **ARE NOT** instructions
- Can be used to control the compilation of some code blocks
- Symbols
- Enumeration type
- Macro definitions

PREPROCESSING - SYMBOLS

- Defined using **#define**

```
#define NMAX 1000
```

```
#define then
```

```
#define BEGIN {
```

```
#define END }
```

```
void main()
```

```
BEGIN
```

```
int vb = 10;
```

```
int vector[NMAX];
```

```
if(vb < NMAX) then printf("smaller");
```

```
else printf("bigger");
```

```
END
```

PREPROCESSING - SYMBOLS

- Each symbol is available:
 - Until the end of the source code;
 - Up to the symbol redefinition;
 - Up the symbol undefinition:
 - `#define NMAX 1000`
 -
 - `#define NMAX 10`
 - ...
 - `#undef NMAX`

PREPROCESSING

- Enumeration type:

```
enum enum_name{symbols list separated by ,}
```

- The symbols form a sequence
- You can explicitly set each symbol value

```
enum engine_type{diesel , gas, gasoline = 5, hybrid = 6}
```

PREPROCESSING

Macro definitions:

```
#define macro_name(symbols list) expression
```

Example:

```
#define SQUARE(X) X*X
```

```
#define ABS(X) (X) < 0 ? - (X) : (X)
```

PREPROCESSING

Macro definitions for generating methods:

```
#define SUMA_GEN(TIP) TIP suma(TIP vb1, TIP vb2) \  
    { return vb1 + vb2; }
```

Conditional compilation:

```
#if expression_1  
    sequence_1  
#elif expression_2  
    sequence_2  
...  
#else  
    sequence_n  
#endif
```

PREPROCESSING

Conditional compilation:

```
#ifdef macro_name
```

```
...
```

```
#else
```

```
...
```

```
#endif
```

or

```
#ifndef macro_name
```

```
...
```

```
#endif
```

PREPROCESSING

Operators # and ##:

- used with #define
- # transforms the next argument into a string “”;

`#define macro1(s) # s`

- ## concatenates 2 elements

`#define macro2(s1, s2) s1 ## s2`

NEW STUFF IN C++ (COMPARED WITH C)

- Console in and out
 - Reading from console: `cin >> variable_name`
 - Printing at console: `cout << variable_name`
- Reserve space in HEAP
 - allocate: `nume_pointer = new tip_data[nr_elemente]`
 - deallocate: `delete [] nume_pointer`
- The & reference type
 - used to define methods input parameters that you want to change : `void Interschimbare(int &a, int &b)`

CLASSES - DEFINITIONS

- Represent structures/entities that contain both data and methods;
- Allow the definition of new data types – ADT (Abstract Data Types);
- Facilitate development of complex solutions;
- Facilitate code reuse;
- Implement OOP concepts – encapsulation, polymorphism (“one interface, multiple methods”), inheritance

CLASSES - DEFINITIONS

- Each object has data (**attributes/fields**) described by the class definition;
- The class defines methods (**methods/functions**) used by objects; these describe the object **interface**;
- Data is sealed/hidden inside the object and it can be accessed only by its methods/interface – **encapsulation**;
- Objects are created by **instantiating** the class;
- By **abstractization (class definition)** it is decided which are the attributes and methods of the object;
- **object state** is given by its attributes values;
- **object behavior** is given by its methods;
- The concept of **passing a message** to the object is equivalent with calling its method;

CLASSES - DEFINITIONS

```
class ClassName
{
    access_specifier:
        attributes;
        member methods;
    access_specifier:
        attribute;
        member methods;
};
```

CLASSES – ACCESS SPECIFIERS

- Sets the access type for class attributes and methods;
- Its area ends with the class ending `}` or when another specifier is;

```
class Test
{
    public:
        ...
    private:
        ...
    public:
        ...
}
```

CLASSES – ACCESS SPECIFIERS

- **private**
 - Each class has a default private specifier at the beginning – set by the compiler;
 - Private attributes and methods can be accessed only from inside the class (from class methods);
- **protected**
 - Used in class frameworks – inheritance;
 - Protected attributes and methods can be accessed only from inside the parent class and its subclasses;
- **public**
 - Public attributes and methods can be accessed from anywhere;

CLASSES - ATTRIBUTES

- Define the object **state**;
- Are initialized when an object is **created/instantiated**; watch out for VS default value
- Accordingly with encapsulation concept, are defined as **private** and are accessed from outside by the object public interface;
- Define the **memory space** of an object (exception: static attributes)
- Extra types: **constant**, **static**;

CLASSES – CONSTANT ATTRIBUTES

- **YOU CAN'T CHANGE** its **value** after initialization;
- syntax:

```
class Test
```

```
{
```

```
    public:
```

```
        const int attribute_1;
```

```
        const char attribute_2;
```

```
}
```

CLASSES – CONSTANT ATTRIBUTES

- **CAN be initialized** only by constructor initialization list :

```
class Test
{
    public:
        Test( ..., int val_at_1):attribute_1(val_at_1), attribute_2(5)
        {
            ...
        }
};
```

CLASSES – STATIC ATTRIBUTES

- Define generic attributes which are not part of the object structure;
- Are used by all the class objects;
- Are like “global variables” but are part of a class;
- Defining a static attribute is **NOT** creating it (it is just a description);
- **ATTENTION** at initialization (it depends on its scope)

CLASSES – STATIC ATTRIBUTES

- syntax:

```
class Test
```

```
{
```

```
    public:
```

```
        static int vb_1;
```

```
        static char vb_2;
```

```
};
```

CLASSES – STATIC ATTRIBUTES

- their creation and initialization is done outside the class definition – using the **class specifier** (Class_name ::)
- syntax:

```
class Test
```

```
{
```

```
    public:
```

```
        static int vb_1;
```

```
};
```

```
int Test::vb_1;
```

```
//initialization
```

CLASSES – THIS POINTER

- for any class, like `Test`, this pointer has a type equal with `Test *`;
- it's the address of the object that calls the class member method/function;
- all class member functions (BUT not static ones) are receiving this as an implicit argument;
- it's the first calling argument, before the input parameters defined by the programmer;

CLASSES - MEMBER FUNCTIONS

- define the object **interface**;
- allow access to the object attributes – based on **encapsulation** the attributes are usually private ;
- define object **behavior**;
- there are some special functions: **constructor, destructor, copy-constructor**;
- different types: static, inline;

CLASSES - MEMBER FUNCTIONS

- you can define the method body inside the class

```
class Test {  
    void Method( ) { ...};  
};
```

- you can define the method body outside the class using the class specifier **class_name::**

```
class Test {  
    void Method( );  
};  
void Test:: Method( ){...};
```

CLASSES - CONSTRUCTORS

- **main role:** creates space for object internal state;
- **secondary role:** inits the object attributes and other logic operations;
- **types:**
 - implicit
 - with arguments
 - with arguments with default values

CLASSES - CONSTRUCTORS

- have the same name as the class (case-sensitive);
- **DON'T HAVE** explicit return type because they create by default a class type object;
- generally defined as public methods (not always);
- the default constructor is generated by the compiler **IF YOU DON'T** explicitly define any constructor;

CLASSES - CONSTRUCTORS

- definition syntax:

```
class ClassName {  
    public:  
        ClassName( ){...}  
};
```

- calling syntax:

```
void main () {  
    ClassName object_1;           //default constructor  
    ClassName object_2(arguments) //argument based constructor  
}
```

CLASSES - CONSTRUCTORS

```
class Test {
```

```
private:
```

```
    int attribute_1;
```

```
public:
```

```
    ...
```

```
};
```

default constructor:

```
Test( ) {attribute_1 = 0; }
```

argument based constructor

```
Test( int value ) {attribute_1 = value ; }
```

```
Test( int value ): attribute_1(value ) {}
```

CLASSES - CONSTRUCTORS

constructor with arguments with default values:

```
Test ( int value = 0) { attribute_1 = value ; }
```

or using the constructor initialization list

```
Test ( int value = 0): attribute_1(value) { }
```

Attention. These constructor replaces the previous versions.

CLASSES - CONSTRUCTORS

constructor with one argument – special case

```
class Test {  
    private:  
        int vb;  
    public:  
        Test(int z) {vb = z;}  
};  
  
void main() {  
    Test t = 34;  
}
```

CLASSES - DESTRUCTOR

- **main role:** frees the space required by an object;
- have the same name as the class; in order to differentiate them from a constructor, their name is prefixed by `~`;
- **DON'T HAVE** return type; implicitly is void;

CLASSES - DESTRUCTOR

- defined as public methods;
- the compiler generates an implicit form (prone to generate memory leaks) if the programmer doesn't define it;
- are **called implicitly** when an object needs to be destroyed;
- stack objects (function locals) are destroyed in reverse order to their initialization;

CLASSES - DESTRUCTOR

- syntax:

```
class Class_name {  
    public:  
        ~Class_name( ){...}  
};
```

- called implicitly at the end of main:

```
void main () {  
    Class_name object_1;  
}
```

CLASSES - DESTRUCTOR

Attention! For dynamically assigned attributes in constructor functions, it is **MANDATORY** to de-allocate them in the destructor. Otherwise, the program generates **memory leaks**.

CLASSES – STATIC METHODS

- Represent functions that do not belong to an object;
- are used by all objects of the class;
- represent "*global functions*" belonging to a class of objects;
- have access only to other static members of the class;
- are called by the **class specifier ::**
- **DO NOT** get the this pointer in the parameter list;

CLASSES – STATIC METHODS

Static methods:

- **syntax:**

```
class Class_Name {  
    public:  
        static void Method_1( ){...}  
};  
  
void main( ) {  
    Class_Name::Method_1( );  
}
```

CLASSES – METHODS

Inline methods:

- short functions that are not called;
- during compiling, the inline function call is replaced by its code, similar to macro functions;
- allow quick code execution by avoiding the effort required for a function call;
- contributes to increase of the executable code size;

CLASSES – METHODS

Inline methods:

- default methods whose body is defined in the class are considered inline (NOT a generic rule, depending greatly on the compiler);
- explicitly, a method is defined as inline by using the **inline** keyword;

```
class Test {  
    void Method( );  
};  
inline void Test:: Method( ){...};
```

CLASSES – METHODS

“accessor” methods:

- allow **read** / **write** access to the private attributes of the class;
- Implement validation of input data;
- are defined in the public area;
- unofficially, reading methods are prefixed with **get** and the modifiers are prefixed with **set**;

CLASSES – METHODS

“accessor” methods:

```
class Class_Name {  
    private:  
        int Attribute_1;  
    public:  
        int Get_Attribute_1( ) { return Attribute_1;}  
        void Set_Attribut_1(int val) {  
            //validate val  
            Atribut_1 = val;  
            //for invalid val throw an exception  
        }  
};
```

CLASSES – METHODS

Passing arguments to functions:

- By value (**Attention** for the **copy-constructor** and **operator=**)

```
class Class_Name {  
    ...  
};  
Class_Name Method1 (Class_Name object);
```

- By reference (**Attention** to changes + return) ;
void Metoda2 (Class_Name & obiect);

- By pointer (**Attention** to changes + return) ;
void Metoda3 (Class_Name * obiect);

CLASSES – COPY CONSTRUCTOR

- **primary role:** assigning space to an object and **initializing it with the values of an existing object;**
- **has a default form** that copies **bit by bit** the value of the existing object in the memory area of the newly created object;
- is automatically called in all cases when an object is defined and initialized;

ClassName object = existingObject

CLASSES – COPY CONSTRUCTOR

- **syntax:**

```
class ClassName{  
    public:
```

```
        ClassName(ClassName & existingObject){...}  
    };
```

- **explicit call:**

```
void main () {  
    ClassName object_1(...);    //constructor  
    ClassName object_2 = object_1;    //copy constructor  
}
```

CLASSES – COPY CONSTRUCTOR

- **Implicit call:** the compiler automatically calls the copy constructor to copy the values of the objects in the parameter list (if sent by value) to the function stack;
- **Implicit call:** the compiler automatically calls the copy constructor to copy the value of the object returned by the function (if returned by value) to the calling program stack;

CLASSES – COPY CONSTRUCTOR

```
class Test {  
public:
```

```
    Test (Test & existent_ob){...}
```

```
    void Method1(Test ob1, Test *ob2) {...}
```

```
    Test Method2(Test ob1) {...}
```

```
};
```

```
void main () {
```

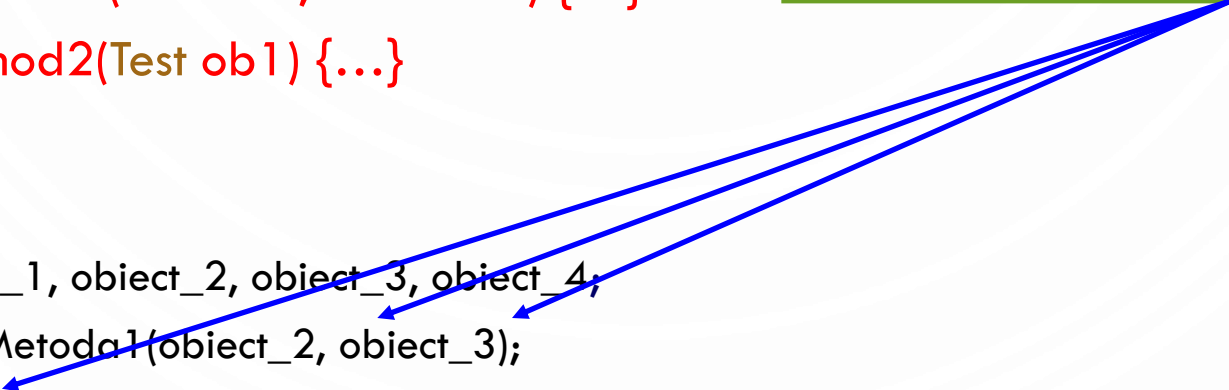
```
    Test object_1, object_2, object_3, object_4;
```

```
    object_1.Method1(object_2, object_3);
```

```
    object_4 = object_1.Method1(object_2);
```

```
}
```

implicit call of the copy constructor



CLASSES – OVERLOADING OPERATOR =

- Primary role: Bitwise copy the value of the source memory area in the destination memory area (the two areas have the same size);
- For objects, copies the value of the source object to the destination object
- Overloaded mandatory by a class member function

CLASSES – OVERLOADING OPERATOR =

- explicit call:

```
class Nume_clasa {
```

```
    ...
```

```
};
```

```
void main () {
```

```
    Nume_clasa object_1(...);
```

```
    Nume_clasa object_2(...);
```

```
    object_2 = object_1;
```

```
}
```

CLASSES – OVERLOADING OPERATOR =

```
class MyClass {  
    MyClass operator = (MyClass object)  
    {  
        //copy values from object to this  
    }  
};  
  
void main () {  
    MyClass object_1(...);  
    MyClass object_2(...);  
    object_2 = object_1;           //call to operator =  
}
```

CLASSES – NESTED CLASSES

- Nested classes are defined within other classes;

```
class OuterClass {  
    ...  
    class NestedClass {...};  
};
```

- The definition is visible only inside the parent class
- access to the child class is only possible through the parent class specifier

```
OuterClass::NestedClass test;
```

CLASSES – FRIEND CLASSES

- Allow access to the private or protected area outside of the class (within the classroom);
- friend class is announced in the class protected by the friend attribute

```
class MyClass{  
    ...  
    friend class SecondClass;  
};  
class SecondClass{  
    ...  
};
```

CLASSES – POINTERS TO MEMBER ATTRIBUTES

- represents the address of an attribute inside the object – offset
- definition :

```
attribute_type MyClass:: * attribute_pointer ;
```

- initialization:

```
attribute_pointer = & MyClass :: attribute_name ;
```

- usage:

```
MyClass object, *pointer_object = &object;
```

```
attribute_type variable = object.* attribute_pointer
```

```
attribute_type variable = pointer_object -> * attribute_pointer
```

CLASSES – POINTERS TO MEMBER METHODS

- represents the address of a method inside the object – offset
- definition:

```
return_type (Class_Name:: * method_pointer) (params) ;
```

- initialization:

```
method_pointer = & Class_Name::method_name ;
```

- usage:

```
Nume_clasa object, *pointer_object = &object;
```

```
return_type variabila = (object.* method_pointer )(params)
```

```
return_type variabila = (pobject->*method_pointer )(params)
```

CLASSES – OVERLOADING FUNCTIONS

- implements the **polymorphism** concept (same thing, multiple interpretations)
- assigning a symbol (function name) several meanings;
- the difference is made according to the function signature = number and type of parameters;
- the returned type is **NOT** a call selection criterion

CLASSES – OVERLOADING FUNCTIONS

```
int sum(int a, int b)
{
    return a + b;
}
```

compiler error
←→
ambiguous definition

```
double sum(int c, int d)
{
    return c + d;
}
```

```
int sum(int a, int b, int c)
{
    return a + b + c;
}
```

CLASSES – OVERLOADING METHODS

- stages identifying function form:
 - identifying the exact form;
 - applying non-destructive conversions on parameters;
 - applying degrading conversions to parameters;
 - application explicitly defined programmer conversions by overloading the cast operator;
 - ambiguous error: overloaded function differs only by return type from ...

CLASSES – OVERLOADING METHODS

```
int sum(int a, int b)
{
    return a + b;
}
```

```
void main()
{
    //identifying the exact form
    int rez1 = sum(5,4);
    //function identification by conversions without loss of values
    int rez2 = sum('0',5); non-destructive
    //function identification by conversions with loss of values (float to integer)
    int rez3 = sum(4.6, 5);
}
```

CLASSES – OVERLOADING OPERATORS

- Operators are implemented by functions:

```
class Test{
```

```
    ...
```

```
};
```

```
void main()
```

```
{
```

```
    Test t1, t2, t3;
```

```
    t1 = t2 + t3;
```

```
}
```

interpretation

`operator+(t1,t2)`

(supraincarcare prin
functie globala)

`t1.operator+(t2)`

(supraincarcare prin
functie membra)

CLASSES – OVERLOADING OPERATORS

- DO NOT change the operators priority;
- DO NOT change associativity;
- preserve cardinality (number of parameters)
- DO NOT create new operators;
- overloaded forms do not automatically compose;
- Do not overload `.* ::?`

CLASSES – OVERLOADING OPERATORS

- Overloading is done through member functions or global function

Exceptions:

- member function: `() [] -> =`
 - global function: `new delete`
- **DON'T** guarantees commutativity;
 - **post** and **pre** forms are overloaded differently;

CLASSES – OVERLOADING OPERATORS

- overloading by member functions or global functions?
- It is a particular operator (a special case) ?
- check the first parameter type:
 - if it has a different type than the class then overload it by a global function
 - if it has the same type as the analyzed class then choose a member function or a global function

CLASSES – OVERLOADING OPERATORS

Operators overloaded by member functions receive the first position as the pointer **this** parameter

```
class Test{  
    Test operator+(Test t, int vb){  
        ...  
    }  
};
```

Attention !

operator + with 3 arguments !!!!!

CLASSES – OVERLOADING

- Care must be taken to choose the returned type:
 - if the operator is called in cascade;
 - if returns object references check if they are not local temporary objects;
 - if returns object values, pay attention to the calls made by the copy constructor;

Attention !

CLASSES – OVERLOADING >> AND <<

- `operator <<` uses `cout` (it's an `ostream&`);

`ostream & operator << (ostream & cout, data_type)`

- `operator >>` uses `cin` (it's an `istream&`)

`istream & operator >> (istream & cin, data_type &)`

- are both overloaded by a global method;

CLASSES – OVERLOADING >> AND <<

```
class Test{
    int info;
    friend ostream& operator << (ostream &, Test);
    friend istream& operator >> (istream &, Test &);
};

ostream& operator << (ostream & iesire, Test t){
    ...
    iesire<<info;
    return iesire;
}

istream& operator >> (istream & intrare, Test & t){
    ...
    intrare>>info;
    return intrare;
}
```

Is defined friend to access private data
IS NOT MANDATORY!

CLASSES – OVERLOADING ++ AND --

- 2 forms: prefix and postfix;
- overloaded by class member function or by global method;

```
int vb1 = 10;  
int vb2 = vb1++;  
int vb3 = 10;  
int vb4 = ++vb3
```

-> vb2 = 10 si vb1 = 11;

-> vb4 = 11 si vb3 = 11

CLASSES – OVERLOADING ++ AND --

```
class Test{
```

```
...
```

```
Test & operator++ ( ) {  
    //.....  
    return *this;  
}
```

pre form by member function

post form by global method

```
}  
friend Test operator++(Test &, int);
```

```
};
```

```
Test operator++ (Test &t, int) {  
    Test copie = t;  
    //prelucrari  
    return copie;  
}
```

is friend to have access on private space
It's not mandatory ! You can use public
accessor methods

CLASSES – OVERLOADING BINARY OPERATORS

- $+$, $-$, $*$, $/$:
- always have 2 parameters;
- the commutativity of the mathematical operation does not make sense in C++ (must be explicitly defined)
- by class member or independent function depending on the operator's form;

CLASSES – OVERLOADING BINARY OPERATORS

- +, -, *, /:
- for object + [object / another type] by class member function

```
class Test{  
    ...  
    int operator+ (int vb) {...}  
};
```

```
void main()  
{  
    Test t;  
    int rez = t + 5;  
}
```

← **X** → int rez = 5 + t;

CLASSES – OVERLOADING BINARY OPERATORS

- +, -, *, /:
- for object + [object / another type] by independent function

```
class Test{  
    ...  
    friend int operator+ (Test,int);  
};
```

```
int operator+ (Test t, int vb){...}
```

CLASSES – OVERLOADING BINARY OPERATORS

- $+$, $-$, $*$, $/$:
- for **another type + object** only by independent method:

```
class Test{  
    ...  
    friend int operator+ (int, Test);  
};
```

```
int operator+ (int vb, Test t){...}
```

CLASSES – OVERLOADING BINARY OPERATORS

- `+=`, `-=`, `*=`, `/=`:
- always have 2 parameters;
- by class member or independent function;

```
class Test{  
    ...  
    friend int operator+= (Test,int);  
};
```

```
int operator+= (Test t, int vb){...}
```

CLASSES – OVERLOADING INDEX OPERATOR []

- always has 2 parameters;
- **only by** class member function;
- is used to allow read / write access on the elements of a string of values in the private area of the object;
- can be called in cascade;
- the index is not mandatory of numeric type (can be a char* or other types);

CLASSES – OVERLOADING INDEX OPERATOR []

```
class Test{  
    int *values;  
    int noValues;  
    ...  
    int operator[ ] (int);  
};
```

Version that only reads the data



```
int Test::operator[ ] (int index){  
    if (index >=0 && index < noValues)  
        return values[index];  
    else throw new exception();  
}
```

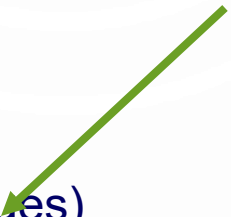
```
void main(){  
    Test t;  
    int vb = t[5];  
    t[3] = 10;  
}
```

//compiler error

CLASSES – OVERLOADING INDEX OPERATOR []

```
class Test{  
    int *values;  
    int noValues;  
    ...  
    int& operator[ ] (int);  
};
```

Version that ensures read /
write data



```
int& Test::operator[ ] (int index){  
    if (index >=0 && index < noValues)  
        return values[index];  
    else throw new exception();  
}  
void main(){  
    Test t;  
    int vb = t[5];  
    t[3] = 10;  
}
```

CLASSES – OVERLOADING CAST OPERATOR

- always has 1 parameter;
- the name of the cast is the type returned;
- has no explicit type returned;
- by member function;
- used to convert between different data tiers;
- **ATTENTION** to the default calls made by the compiler to determine the signature of a function;
- has an **explicit** form

CLASSES – OVERLOADING CAST OPERATOR

```
class Test{  
    int value;  
  
    ...  
    int operator int () { return value;}  
};  
  
void main(){  
    Test t;  
    int vb = t;    //equivalent to vb = t.value;  
}
```



CLASSES – OVERLOADING OPERATOR !

- has one parameter;
- by member or independent function;

```
class Test{  
    int value;  
  
    ...  
    void operator ! () {value *= -1;}  
};  
  
void main(){  
    Test t;  
    !t;  
}
```

CLASSES – OVERLOADING OPERATOR COMMA ,

- has two parameters;
- by member or independent function;
- typically returns the value of the last parameter;

```
class Test{  
    int value;  
    ...  
    Test& operator ! (Test& t) {return t;}  
};  
void main(){  
    Test t1,t2, t3,t4;  
    t4 = (t1,t2,t3);    //equivalent to t4 = t3;  
}
```

CLASSES – OVERLOADING OPERATOR FUNCTION

- overload operator function;
- has a variable number of parameters;
- overloaded by member function;
- does not create a new way to call a function;
- is created an operator function that can receive an arbitrary number of parameters;

CLASSES – OVERLOADING OPERATOR FUNCTION

```
class Test{  
    int value;  
  
    ...  
    int operator () (int i, int j) {  
        valoare = i + j;  
        return valoare;}  
};  
void main(){  
    Test t;  
    t(3,4);  
    int vb = 10 + t(5,10);  
}
```

- operator ()

- object used as a function

CLASSES – OVERLOADING OPERATOR ->

- always has 1 parameter;
- mandatory by member function;
- return pointer to an object on which it operates;

```
class Test{  
    ...  
    Test * operator-> ( ) {return *this;}  
};
```

CLASSES – CONVERTING BETWEEN OBJECTS TYPES

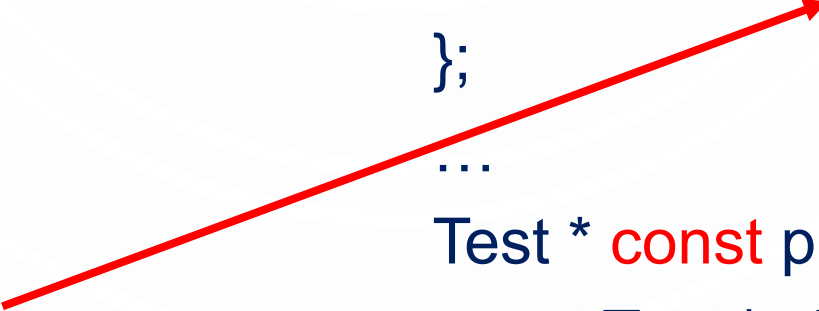
- overloading the constructor of the resulting class;
- overloading the cast operator of the source class;
- the implicit calls of the constructor of the resulting class are removed by the explicit attribute defined in the definition;

CLASSES – CONSTANT POINTERS TO OBJECTS AND POINTERS TO CONSTANT OBJECTS

- definition is made by positioning the attribute **const** in relation to the type and name of the pointer;

```
class Test{  
    ...  
    void Metoda const ( ) {...}  
};  
...  
Test * const pConstantTest;  
const Test * pTestConstant1;  
Test const * pTestConstant2;
```

The object referenced by this is constant



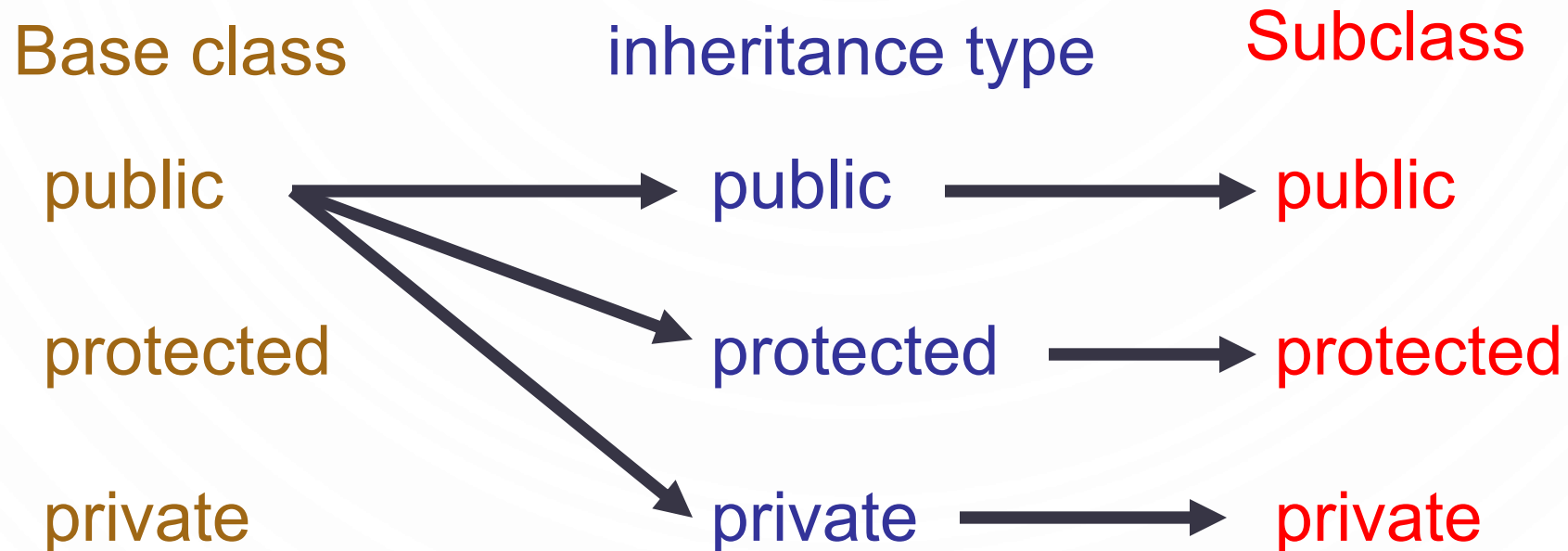
CLASSES – EXTENDING / INHERITANCE

- it's all about reusing code;
- developing new entities (classes) starting from the existing ones
- Extending - the existing class is extended in a new class;
- Inheritance - the newly defined class inherits the attributes + methods of the derived class (of the base class);

```
class BaseClass{  
  
};  
class SubClass: inheritance type BaseClass{  
  
};
```

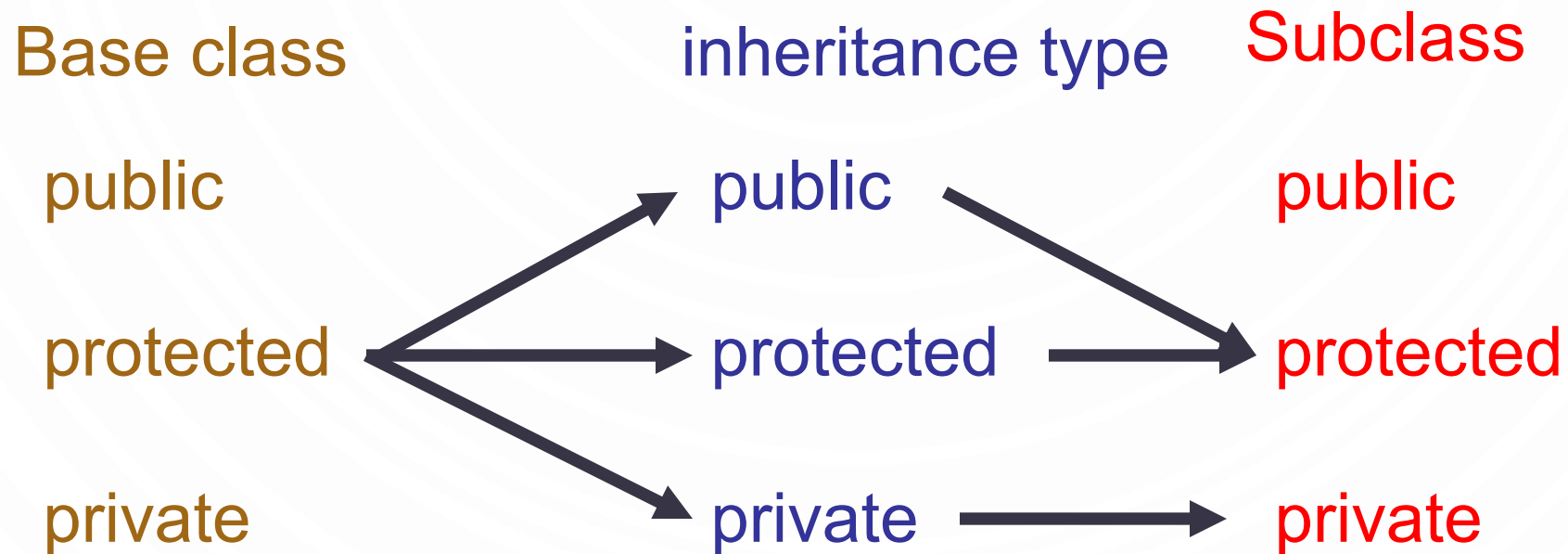
CLASSES – EXTENDING / INHERITANCE

- inheritance DO NOT remove access restrictions in the base class;



CLASSES – EXTENDING / INHERITANCE

- inheritance DO NOT remove access restrictions in the base class;



CLASSES – EXTENDING / INHERITANCE

Base class

inheritance type

Subclass

public

public

protected

protected

private

private

not
accessible

CLASSES – INHERITANCE

- exceptions to the derivation type (protected or private) for certain attributes or methods (public in the base class): making public attributes

```
class Base{  
    public:  
        int attribute1;  
        int attribute2;  
};  
class Subclass: private Base{  
    public:  
        Base::attribute1;  
};
```

Diagram illustrating inheritance rules for attributes:

- From `Base` to `Subclass`:
 - `attribute1` (public in `Base`) becomes `private` in `Subclass` (indicated by an arrow).
 - `attribute2` (public in `Base`) becomes `private` in `Subclass` (indicated by an arrow).
- From `Base` to `Subclass`:
 - `Base::attribute1` (public in `Base`) becomes `public` in `Subclass` (indicated by an arrow).

CLASSES – INHERITANCE

- prin derivare noua clasa primește de la clasa de baza toate metodele + attributele

```
class Base{
```

```
    int attribute1;
```

```
    int attribute2;
```

```
};
```

```
class Subclass : private Base{
```

```
    int new_attribute;
```

```
};
```



inheritance
↓



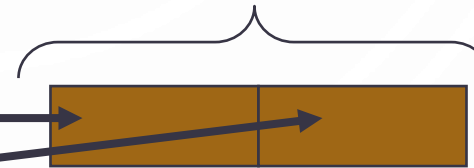
CLASSES – INHERITANCE

- each constructor is solely responsible for the area of the class it represents

```
class Base{  
    int attribute1;  
    int attribute2;  
};
```

```
class Subclass: private Base{  
    int new_attribute;  
};
```

Base class constructor



mostenire



Subclass constructor



Base class constructor

CLASSES – INHERITANCE

- subclass object is built by BASE CONSTRUCTOR + SUBCLASS CONSTRUCTOR

```
class Base{  
    Base(){...}  
    Base(param list){...}  
};
```

implicit call to **Base()**

explicit call
:Base(param list)

```
class Subclass : inheritance type Base{  
    Subclass(){...};  
    SAU  
    Subclass() : Base(param list) {...}  
};
```

CLASSES – INHERITANCE

- destroying subclass object = SUBCLASS DESTRUCTOR + BASE DESTRUCTOR
- **ATTENTION !** each destructor must focus strictly on what the class constructors did

```
class Base{  
    int * memory();  
    ~Baza(){delete [ ]memory;}  
};  
class Subclass : inheritance type Base{  
    ~Subclass(){delete [ ]memory};  
};
```

2 – releasing memory !!!!!!!
(does it exists ???)

1- releasing memory

It's wrong !!!!!!!

CLASSES – INHERITANCE

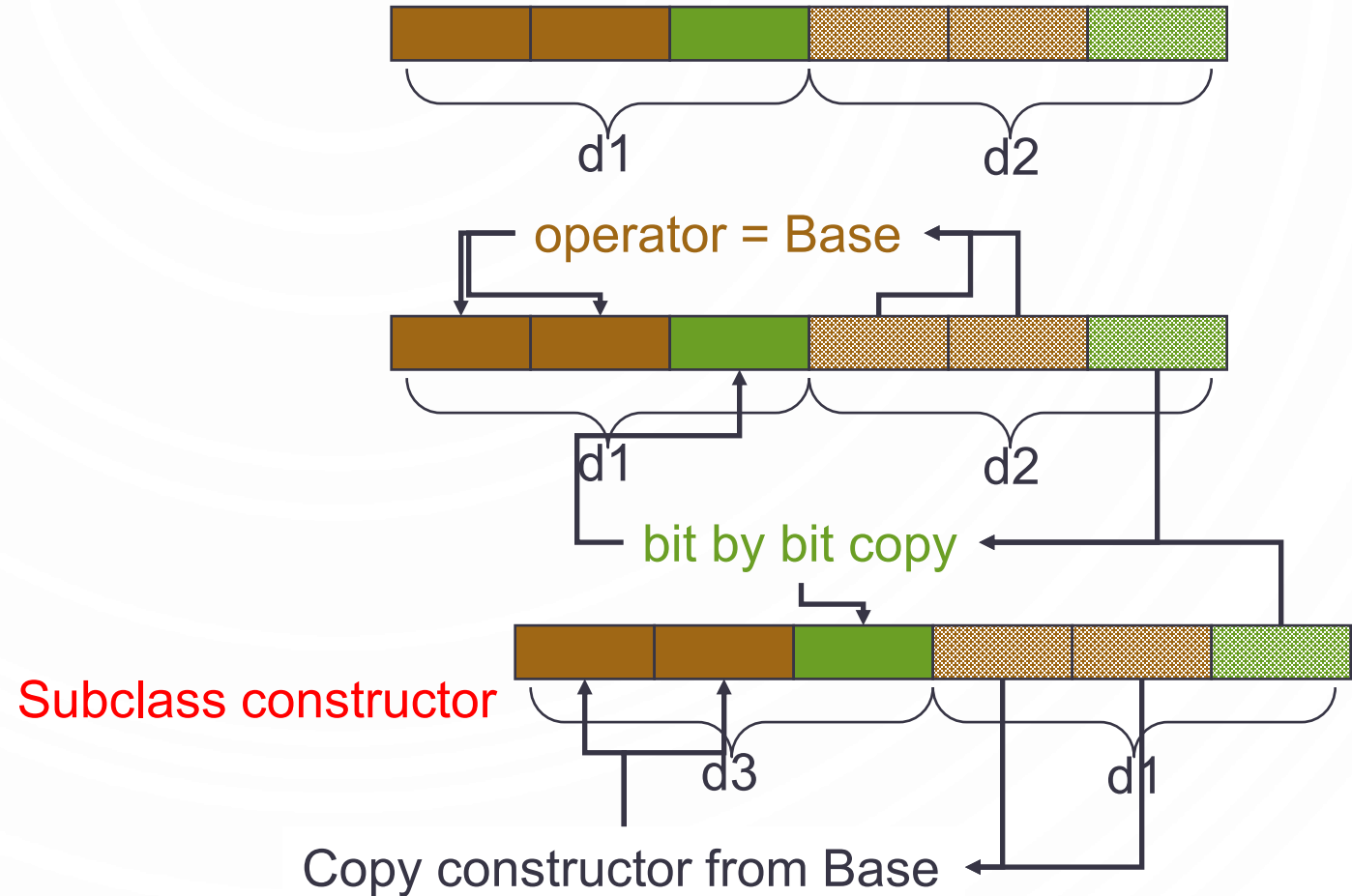
Methods that are not fully inherited:

- operator = and Copy Constructor

```
class Base{  
    int atribut1;int atribut2;  
    Base& operator=(Base& b){...}  
    Base(Base& b) {...}  
};  
class Subclass : private Base{  
    int new_attribute;  
};
```

CLASSES – INHERITANCE

```
void main(){  
    Subclass d1;  
    Subclass d2;  
  
    d1 = d2;  
  
    Subclass d3 = d1;  
}
```



CLASSES – INHERITANCE & UPCASTING

- UPCASTING – it is allowed to implicitly convert objects or pointers of the derived type into objects or base type pointers

```
class Base{
```

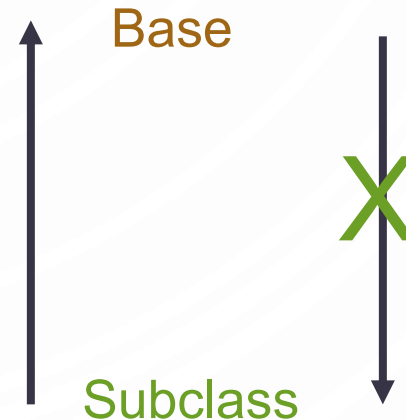
```
    ...
```

```
};
```

```
class Subclass: public Base{
```

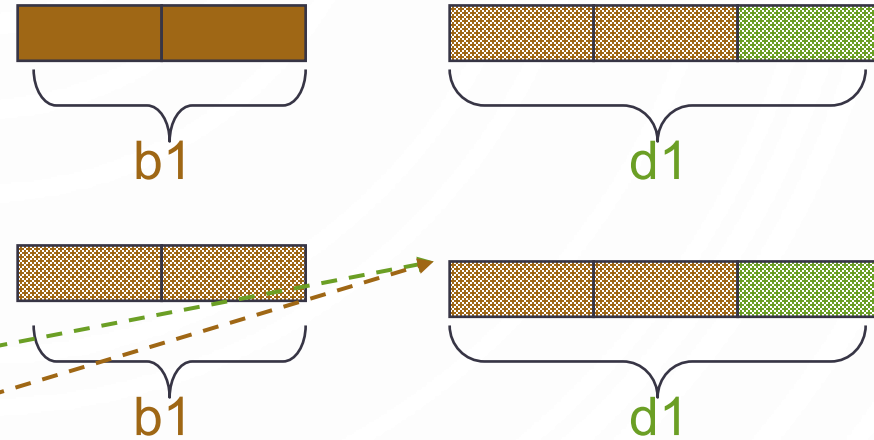
```
    ...
```

```
};
```



CLASSES – INHERITANCE & UPCASTING

```
void main(){  
    Subclass d1, *pd1;  
    Base b1, *pb1;  
  
    b1 = d1;  
  
    pd1 = &d1;  
    pb1 = pd1;  
}
```



CLASSES – INHERITANCE & UPCASTING

- can be defined functions with the same header in the base class and the derived class - OVERRIDING

```
class Base{  
    int Method1(int a){...}  
};
```

```
class Subclass: private Base{  
    int new_attribute;  
    int Method1(int a){...}  
};
```

```
void main(){  
    Subclass d1;
```

```
    d1.Method1(5);
```

```
    d1.Base::Method1(5);  
}
```

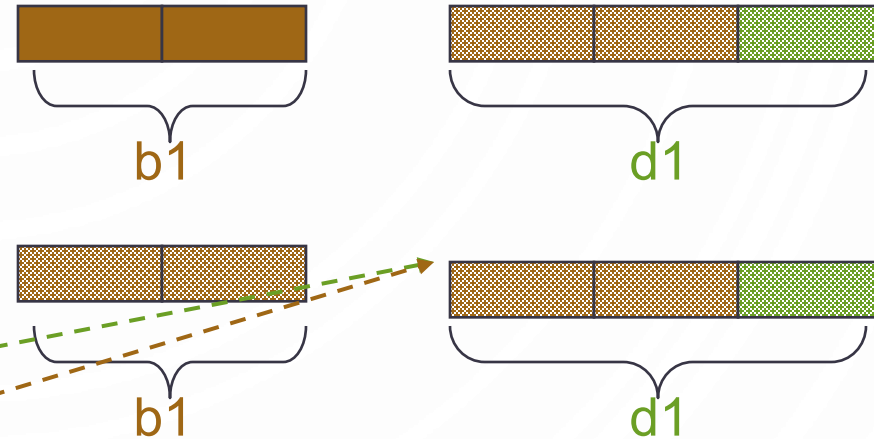
CLASSES – INHERITANCE & UPCASTING

```
void main(){  
    Subclass d1, *pd1;  
    Base b1, *pb1;
```

```
    b1 = d1;
```

```
    pd1 = &d1;  
    pb1 = pd1;
```

```
    b1.Method1(5);  
    pb1->Method1(5);  
}
```



Always the
method from
Base;

CLASSES – INHERITANCE & UPCASTING

- UPCASTING + redefine methods
- the version of the function is established from compile time (early binding),
- regardless of whether UPCASTING is achieved by values or pointers, the base class method is always called

```
void main(){  
    Subclass d1, *pd1;  
    Base b1, *pb1;  
    b1 = d1;  
    pd1 = &d1;  
    pb1 = pd1;  
}
```

CLASSES – INHERITANCE & VIRTUAL METHODS

- VIRTUAL methods:
- allow overriding the base class methods in the derived class

```
class Base{  
    virtual int Method1(int a){...}  
};
```

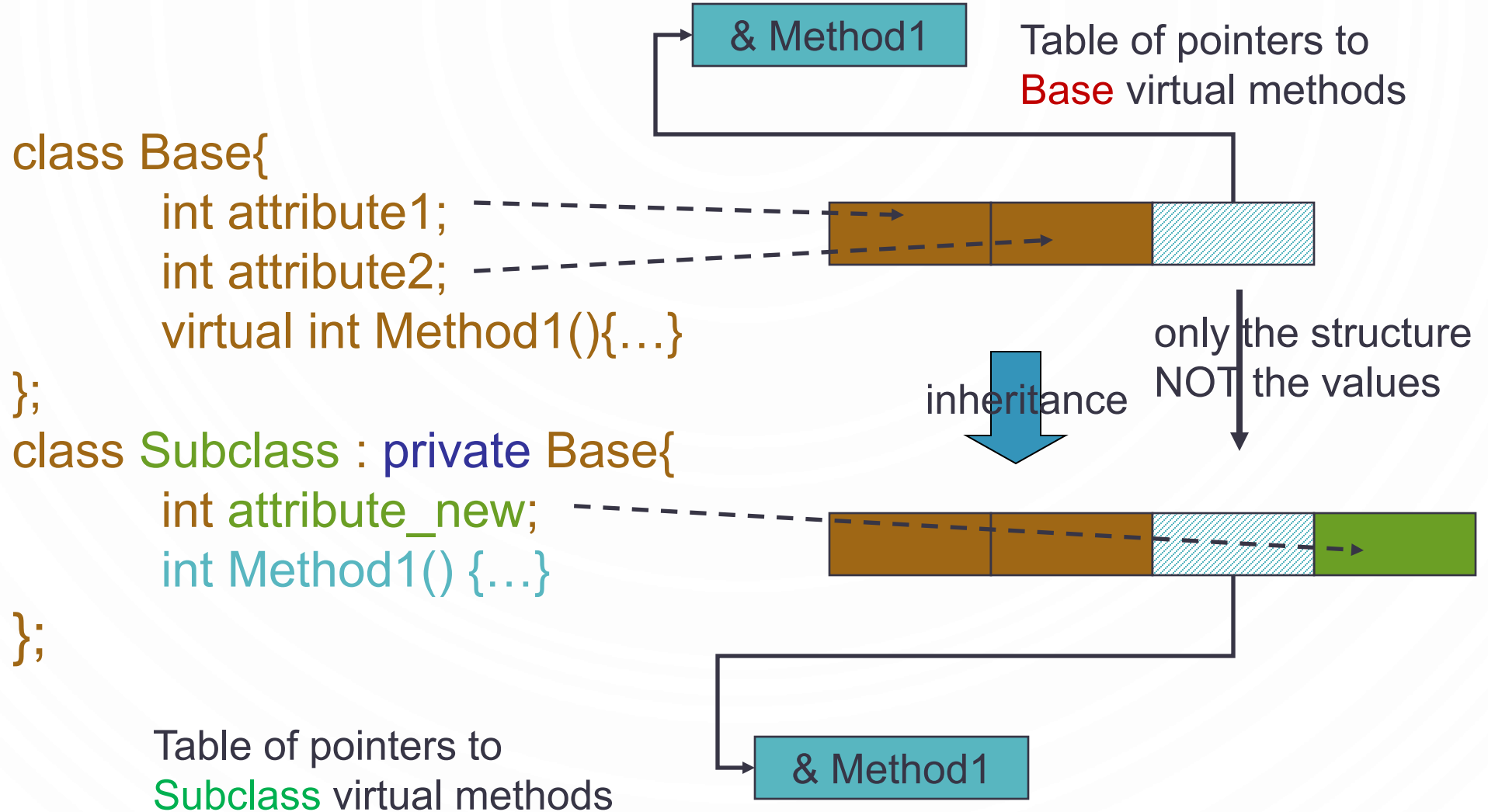
```
class Subclass: private Base{  
    int new_attribute;  
    int Method1(int a){...}  
};
```



CLASSES – INHERITANCE & VIRTUAL METHODS

- VIRTUAL functions:
- the version of the function is determined at the time of execution (late binding)
- each class contains a table of pointers to virtual functions (_vtptr);
- each object receives a pointer to the pointer table at virtual functions
- if UPCASTING is done by pointers (NOT by values), it is called the method in the derived class

CLASSES – INHERITANCE & VIRTUAL METHODS



CLASSES – INHERITANCE & VIRTUAL METHODS

```
void main(){  
    Subclass d1, *pd1;  
    Baza b1, *pb1;
```

```
    b1 = d1;  
    b1.Method1(5);
```

```
    pd1 = &d1;  
    pb1 = pd1;
```

```
    pb1->Method1(5);
```

```
}
```

ALWAYS the
Base version;

the Subclass
version for the
Method1 virtual;

CLASSES – INHERITANCE & VIRTUAL METHODS

- the virtual nature of a function is inherited
- the function on the last level where she was redefined is responsible for her subierarchy;
- the function becomes and remains virtual from its first definition in the hierarchy of which is announced as virtual
- **ATTENTION** to virtual destructors

CLASSES – INHERITANCE & VIRTUAL METHODS

POLYMORPHISM (same thing, multiple interpretations) :

- **OVERLOADING** functions in the same class
- **OVERRIDING** virtual methods in subclasses

CLASSES – INHERITANCE VS COMPOSITION

```
class Vehicle{  
    ...  
};
```

```
class Automobile : public Vehicle{  
    ...  
};
```

is implemented
when between the
derived class and
the base class
there is a *is a*
relation;

CLASSES – INHERITANCE VS COMPOSITION

```
class Engine{  
    ...  
};  
  
class Automobile{  
    Engine engine;  
};
```

is implemented
when there is a
has a relationship
between the main
class and the
included one;

CLASSES – MULTIPLE INHERITANCE

- builders are called in the **order of derivation**;
- destructors are invoked in reverse order;
- **ambiguities** in addressing inherited members who are called the same in the parent classes
- **ambiguities** for inheritance from base classes with a common parent

CLASSES – MULTIPLE INHERITANCE

```
class Base1{  
};  
class Base2{  
};
```

```
class Subclass: public Base1, public Base2{  
};
```

CLASSES – VIRTUAL PURE/ABSTRACT METHODS

- virtual functions that do not have a body defined in the class in which they are announced
- are defined by expression

```
virtual return_type method_name( params) = 0;
```

- IMPOSE the overriding of the function in the derived class (if it is not desired to make abstract the derived class)

CLASSES – VIRTUAL PURE/ABSTRACT METHODS

```
class Abstract_class{  
    virtual int Method1(int a) = 0  
};
```

```
class Subclass : public Abstract_class{  
    int Method1(int a){...}  
};
```

CLASSES – ABSTRACT CLASSES

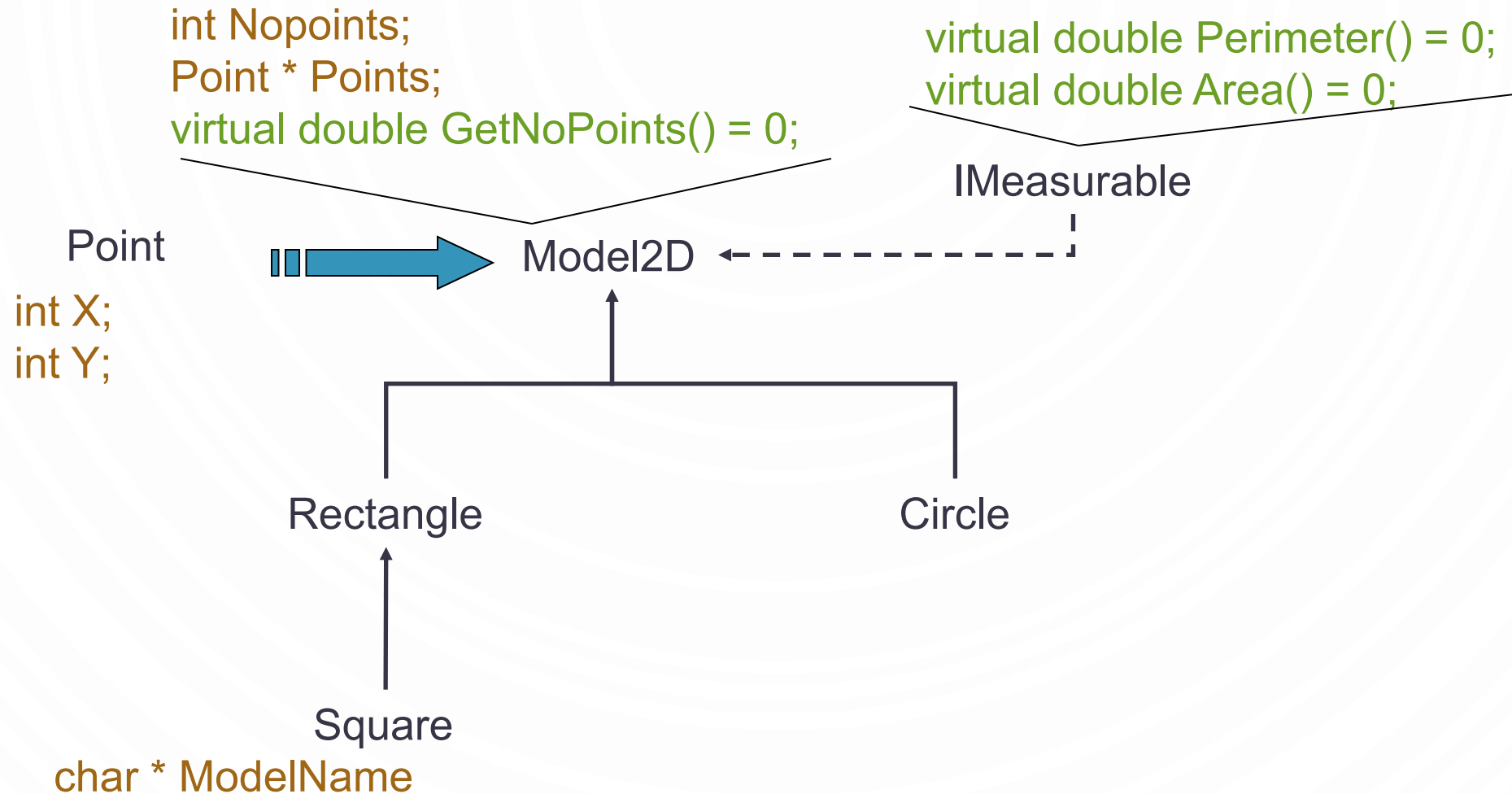
- classes that contain at least a pure virtual function;
- has an interface role for classes that need to define a set of common methods
- a contract between owners of several classes requiring the definition of a series of common methods;
- the contract is implemented by inheriting the abstract class;

CLASSES – ABSTRACT CLASSES

- YOU CAN'T instantiate abstract classes;
- used as support for inheritance

```
class Abstract_Class{  
    int attribute1;  
    virtual int Method1(int a) = 0  
};  
void main(){  
    Abstract_Class ab;  
}
```

CLASSES – ABSTRACT CLASSES



CLASSES – ABSTRACT CLASSES

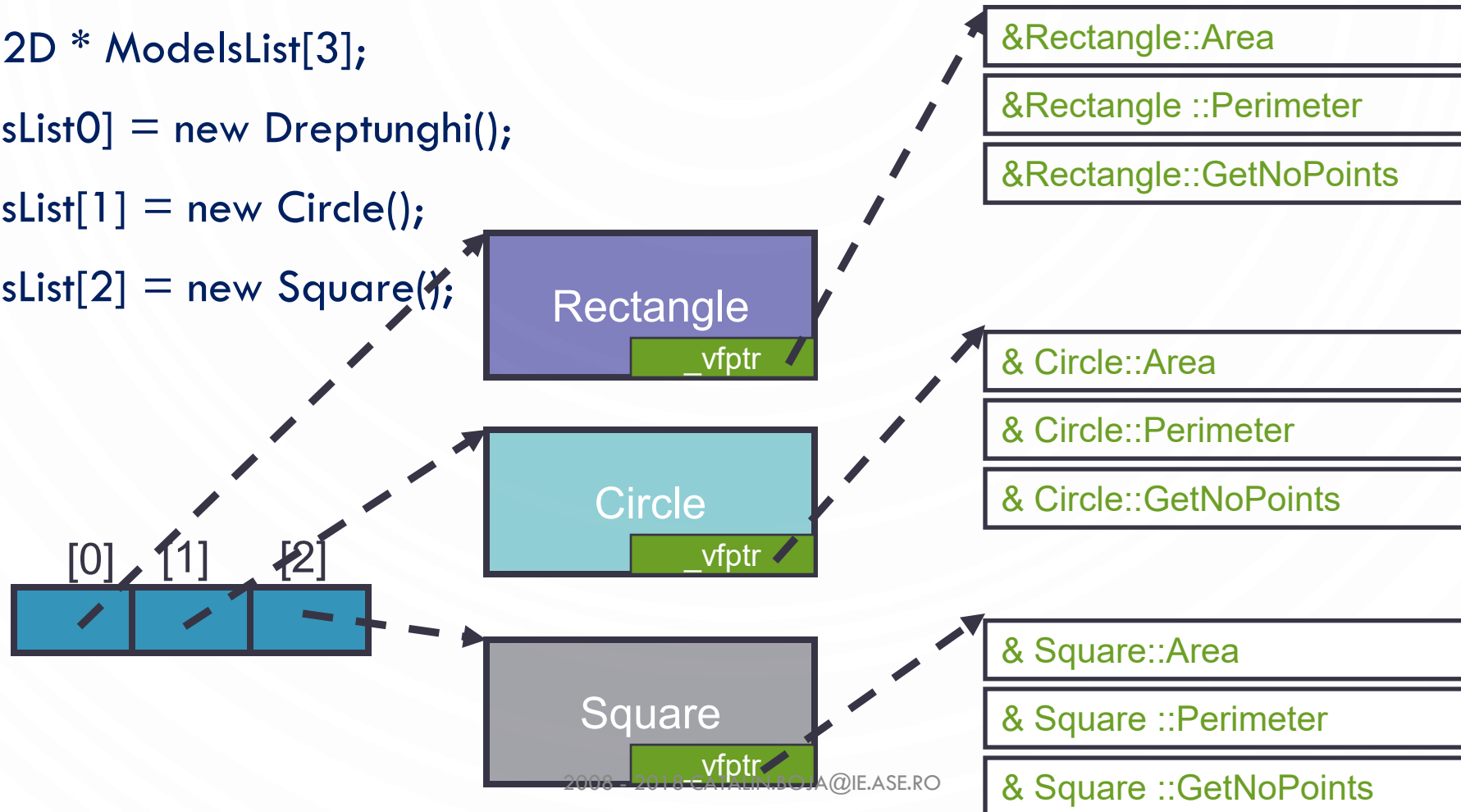
Simple hierarchy

```
Model2D * ModelsList[3];
```

```
ModelsList[0] = new Dreptunghi();
```

```
ModelsList[1] = new Circle();
```

```
ModelsList[2] = new Square();
```



CLASSES – NAME SPACES

- is a way of grouping global variables, classes and global functions
- allows the definition of classes, variables, functions with identical names but in different namespace
- facilitates distributed code development since programmers do not impose name restrictions between them
- keyword **namespace**

CLASSES – NAME SPACES

- Definition:

```
namespace Collection1 {  
    //definitions of classes, variables and functions  
};
```

- Alias definition:

```
namespace Collection = Collection1 ;
```

- Adding elements:

```
namespace Collection1 {  
    int vb1;  
};  
namespace Collection2 {  
    int vb2;  
};
```

CLASSES – NAME SPACES

- Usage – using namespace name:

```
namespace Collection1{  
    int vb1;  
};  
namespace Collection2{  
    int vb2;  
};  
void main(){  
    Collection1::vb1 = 10;  
    Collection2::vb2 = 20;  
}
```

CLASSES – NAME SPACES

- Usage – with `using namespace`:

```
namespace Collection1{
```

```
    int vb1;};
```

```
namespace Collection2{
```

```
    int vb2;};
```

```
void main(){
```

```
    using namespace Collection1;
```

```
    vb1 = 10;
```

```
    Collection2 ::vb2 = 20;
```

```
}
```

CLASSES – EXCEPTIONS HANDLING

- exception – processing of certain input data is not managed or is not possible (eg division at 0, reading outside a massive)
- allows management of the exceptional situations that lead to the immediate termination of the program
- needed to deliver robust and reliable programs
- implemented by try, catch and throw
- allows management of system errors and bugs defined by the programmer

CLASSES – EXCEPTIONS HANDLING

```
try {  
    //code sequence  
    if(conditie_1) throw exception;  
    //code sequence  
    if(conditie_2) throw general_exception;  
}  
catch(exception){ //specific code sequence  
  
catch(other_exception) { //specific code sequence  
  
catch(...){ //generic code sequence
```

The diagram consists of two dashed arrows. The first arrow starts from the right side of the try block and points towards the first catch(exception) block. The second arrow starts from the right side of the try block and points towards the catch(other_exception) block.

CLASSES – EXCEPTIONS HANDLING

`try{...}`

- contains the sequence that generates exceptions using throw;
- has at least one catch block associated
- between the try block and the associated catch blocks there are no other instructions

`catch( exception type)`

- manages an exception of a specific/generic type

`catch( ...)`

- Manages all exceptions (no handled by other more specific catch blocks)

CLASSES – EXCEPTIONS HANDLING

Catch blocks are defined in increasing order of the managed exceptions generality

```
try { ... }
```

```
catch(exception_type_1){...}
```

```
catch(exception_type_2){...}
```

```
...
```

```
catch(...){...}
```

CLASSES – EXCEPTIONS HANDLING

`catch(...){...}` can be replaced by the standard function called to treat an unmatched exception – `terminate( )`

```
void function_terminate(){  
    cout << "function_terminate()";  
    exit(-1);  
}  
set_terminate( function_terminate );
```

CLASSES – EXCEPTIONS HANDLING

Defining Methods (convention, **NOT** rule) that generate exceptions:

- function announces by header what exceptions it generates

void function() throw(exception, DivideByZero){...}

- the function can generate any type of exception

void function(){...}

- the function does not generate exceptions

void function() throw(){...}

CLASSES – EXCEPTIONS ADVANTAGES

- allows separation of error handling processing;
- a new method of announcing the successful execution or not of a function (to the detriment of the global variables)
- **Important** for managing bugs in builders/constructors

CLASSES – EXCEPTIONS ADVANTAGES

- the function announces an exception and the calling program:
 - fix the problem
 - decide to call again the function (with other values) or continue the program
 - generates other results
 - end the program in a "normal" way (undo memory, save partial results);
 - partially solve the problem and throw a new exception for a higher context.

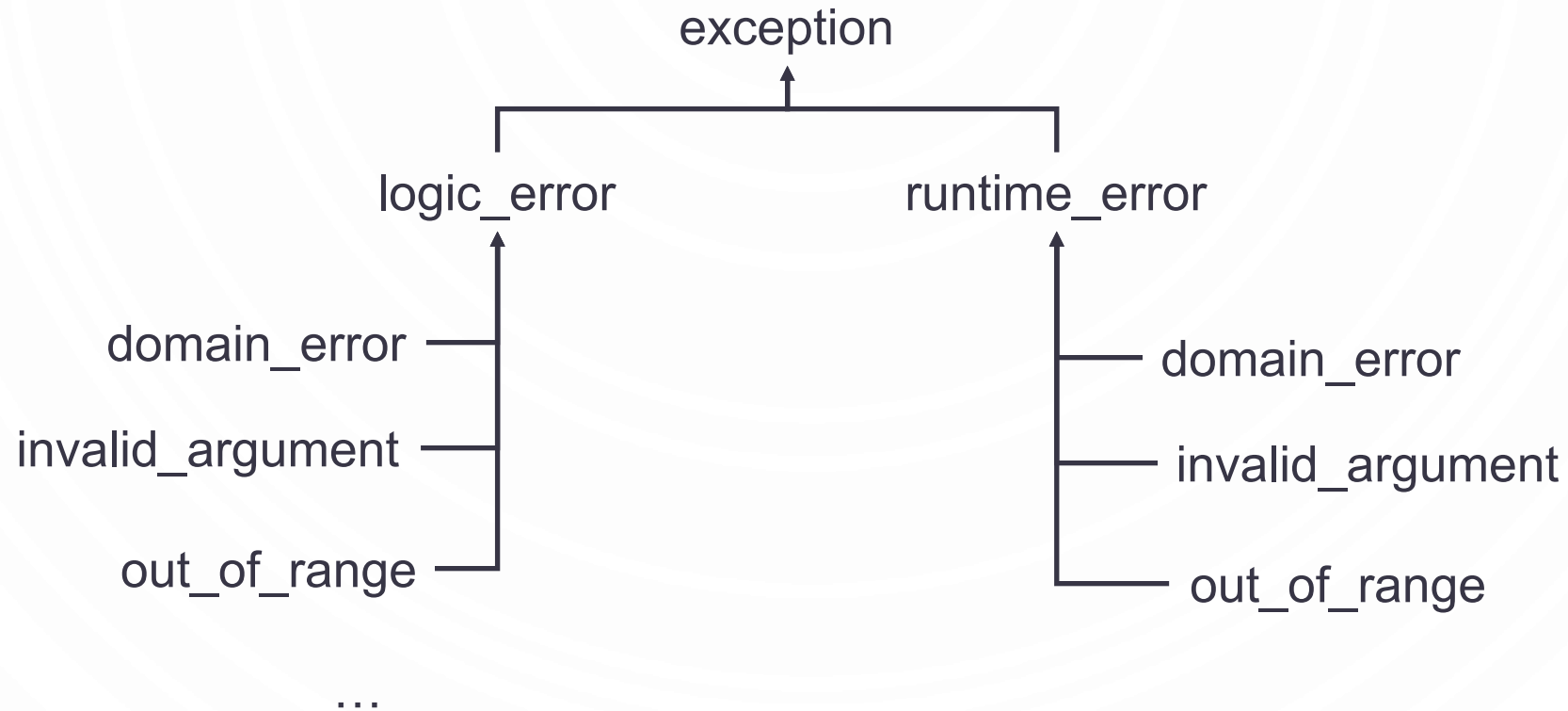
CLASSES – EXCEPTIONS DISADVANTAGES

- may complicate code;
- in C ++ is an alternative to treating local errors (inside the function)
- are inefficient in terms of program execution
- capturing exceptions by value;
- not for asynchronous events (in C ++ the exception and her handler are present in the same call (call stack))
- **Important** generating exceptions in destructor and constructor functions;

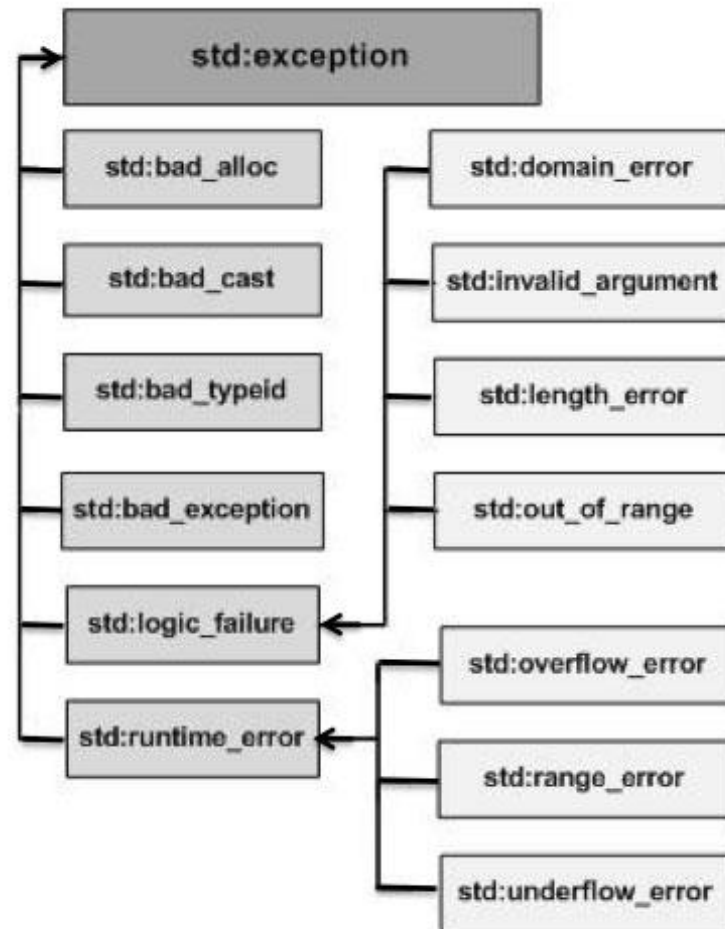
CLASSES – EXCEPTIONS DISADVANTAGES

- **generating exceptions in the constructor** interrupts its execution and the object is no longer built (the destructor is NOT CALLED anymore) but the dynamic allocation done until throw is reached, will generate memory leaks
- **generating exceptions in the destructor** interrupts its execution and can generate memory leaks

CLASSES – EXCEPTIONS



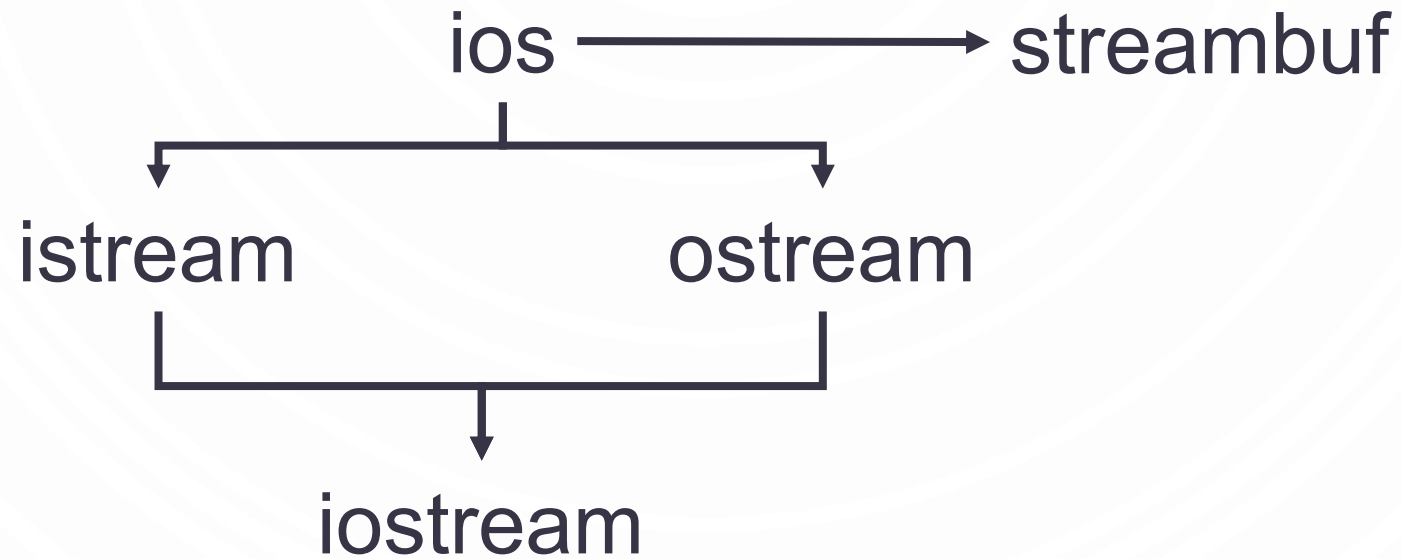
CLASSES – EXCEPTIONS



CLASSES – STREAMS

- **STREAM** – object that allows the management and handling of byte strings
- Used normally with output stream operator << (inserter) and input stream operator >> (extracter)
- Standard objects: **cout** is an **ostream** and **cin** is an **istream**, cerr (for errors standard stream)

CLASSES – STANDARD STREAMS



CLASSES – STANDARD STREAMS

Stream data formatting:

- methods for **cin** and **cout** objects (for example, width, fill)
- manipulators (iomanip.h)
- formatting flags of the ios class (ios :: *setiosflags* method)
- formatting flags (bits) of **cin** and **cout** objects (*setf* method)

CLASSES – STANDARD STREAMS

- dec
- hex
- oct
- setprecision(int)
- endl
- ends
- WS
- flush
- setbase(int)
- setfill()
- setw(int)
- setiosflags(long)
- resetiosflags(long)

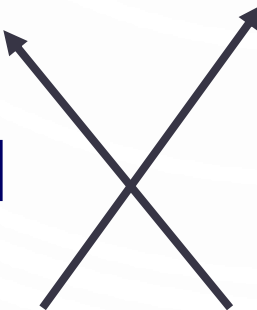
Stream manipulators

CLASSES – STANDARD STREAMS

- `ios::left`
- `ios::right`
- `ios::internal`
- `ios::dec`
- `ios::hex`
- `ios::showpos`
- `ios::showbase`
- `ios::scientific`
- `ios::fixed`
- `ios::showpoint`
- `ios::skipws`
- `ios::stdio`
- `ios::uppercase`
- `ios::unitbuf`

Formatting flags (`setiosflags` and `resetiosflags`):

CLASSES – STANDARD STREAMS

- `ios::basefield`
 - `ios::floatfield`
 - `ios::adjustfield`
 - `ios::dec`
 - `ios::hex`
 - `ios::oct`
 - `ios::fixed`
 - `ios::scientific`
 - `ios::left`
 - `ios::right`
 - `ios::internal`
- 

Formatting flags (`long ios::setf(long val, long ind)`):

CLASSES – STANDARD STREAMS

Read strings with **cin**:

- Same as scanf() function considers a string terminator and space
- for special reads use the **get** and **getline** methods of the **cin** object;

cin.getline(char* adr_buffer, int no_bytes, int delimiter) – removes the delimiter from the input

cin.get(char* adr_buffer, int no_bytes, int delimiter)

CLASSES – STANDARD STREAMS

Read / write error detection:

- errors reported by setting (value 1) some flow status flag (s):
 - failbit
 - badb
- flags are tested using the following methods:
 - `boolean good ()`
 - `int fail ()`
 - `int bad ()`
- flags are reset using the `clear ()` method

CLASSES – STANDARD STREAMS

```
int Cod = 0;
bool IsOk = false;
while(!IsOk){
    cout<<"\n Cod angajat:";
    cin>>Cod;
    IsOk = cin.good();
    if(!IsOk)
        cerr<<"\n Valoare eronata pentru cod !";
    cin.clear();
    cin.ignore(256,'\n');
}
```

reset flags

empty input buffer

CLASSES – STANDARD STREAMS

Read / write error detection:

For errors >> returns NULL

```
int value;
```

```
while(cout<<"Input a value or CTRL+Z:", cin>>value);
```

operator (,) returns last value of the expression

CLASSES – FILES & NONSTANDARD STREAMS

- Using objects defined in `fstream.h`
- **ifstream** - working with input files;
- **ofstream** - work with output files;
- **fstream** - work with input / output files;
- Created by:
 - Different constructors:
`fileStream fileObject( char * fileName)`
 - Using `open()`:
`fileObject.open (char * fileName, long openMode)`

CLASSES – FILES

Open mode:

- `ios::in` - read mode
- `ios::out` - write mode
- `ios::ate` - open and place the file cursor at the end
- `ios::app` - open for appending
- `ios::trunc` - open and overwrite (delete previous content)
- `ios::nocreate` - don't open if doesn't exist
- `ios::noreplace` - don't open if exists
- `ios::binary` - open in binary mode

CLASSES – FILES

Read / Write from files:

- Using `>>` and `<<`;
- Using `get( )` (returns EOF, -1, for end of file) and `put ( )` works at byte level
- Using `read( )` (returns NULL, for end of file) and `write( )` :

`istream& istream::read(char * buffer, int nr_octeti)`

`ostream& ostream::write(char * buffer, int nr_octeti)`

CLASSES – FILES

Read / Write from files:

RAM buffer into which you write
bytes from a file

`istream& istream::read(char * buffer, int noBytes)`

RAM buffer from which you
write bytes into a file

`ostream& ostream::write(char * buffer, int noBytes)`

No of bytes to read/write

CLASSES – FILES

Read from files:

- end-of-file testing is also done by checking the eofbit flag by the method of a fstream object

`bool eof()`

CLASSES – FILES

Get a location in a file:

- For input files (read mode):

`istream & istream::seekg(long streamoff, ios::seekdir)`

or

`istream & istream::seekg(long streampos)`

where:

`ios::seekdir` is one of the following flags:

`ios::beg` – beginning of the file

`ios::cur` – current location

`ios::end` – end of the file

CLASSES – FILES

Get a location in a file:

- For output (write):

`ostream & ostream::seekp(long streamoff, ios::seekdir)`

or

`ostream & ostream::seekp(long streampos)`

where:

`ios::seekdir` may be:

`ios::beg` – beginning of the file

`ios::cur` – current location

`ios::end` – end of the file

CLASSES – NONSTANDARD STREAMS

Positioning in files:

- for output files (writing) the determination of the current position is done by the **long tellp ()** method returning the number of bytes at the beginning of the file
- for input files (reading) the determination of the current position is done by the **long tellg ()** method returning the number of bytes at the beginning of the file

CLASSES – NONSTANDARD STREAMS

Recap files from your previous courses:

- Sequential organization with fixed and variable length records
- Direct access
- Indexed files
- Inverse files

CLASSES –TEMPLATE FUNCTIONS

- allow increasing the degree of generalization by defining function templates
- the generic types are used to define:

`class T`
`typename T`

- the function is instantiated by the compiler to use when the generic type is replaced by a specific type

CLASSES – TEMPLATE FUNCTIONS

- definition:

```
template <typename T1, typename T2, ...>
```

```
returned_type function_name( T1 param1, T1 param2, T2 param3, ... )
```

- init & usage:

```
function_name <real_type, real_type, ...>
```

```
( param1, param2, param3,... )
```

CLASSES – TEMPLATE FUNCTIONS

- definition:

```
template <typename T>
```

```
T add (T a, T b){ return a+b;}
```

- init & usage:

```
int sum = add<int> (5,6);
```

```
double sum2 = add<double>(5.5, 6.6);
```

CLASSES – TEMPLATE CLASSES

- represent a parameterized descriptions of the class;
- allow for adaptation to concrete data types (fundamental + user types)
- by instantiating the template, the compiler generates concrete classes

CLASSES – TEMPLATE CLASSES

```
template <class T1, typename T2, ..., type1 c1, type2 c2, ...>
```

```
class class_name{
```

```
    ...
```

```
}
```

Existing type

```
class_name <tipc1, tipc2, ..., val1, val2, ...> object;
```

CLASSES – TEMPLATE CLASSES

```
template <typename T>
```

```
class Vector{
```

```
    T * values;
```

```
    int dimension;
```

```
public:
```

```
    ...
```

```
};
```

```
Vector<int> v1;
```

CLASSES – TEMPLATE CLASSES

- A template definition can contain constant values

```
template <typename T, int n>
```

```
class Vector_S{
```

```
    T valori[n];
```

```
    int dim;
```

```
public:
```

```
    ...
```

```
};
```

```
Vector<int,10> v1;
```

CLASSES – TEMPLATE CLASSES

```
template <typename T=int, int n=5>
```

```
class Vector_S{
```

```
    T valori[n];
```

```
    int dim;
```

```
public:
```

```
    ...
```

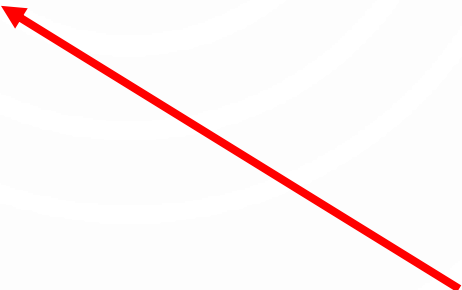
```
};
```

```
Vector<int,10> v1;
```

```
Vector<> v2;
```

```
Vector<double> v2;
```

Default values for templates



CLASSES – TEMPLATE CLASSES

Implementation scenarios:

- Want to use a template or a concrete case?
 - concrete case:

```
int compare (Vector <int> v1, Vector <int> v2) {...}
```

- template:

```
template <typename T1, typename T2> int compare (Vector <T1> v1, Vector  
<T2> v2) {...}
```

CLASSES – TEMPLATE CLASSES

Using template classes in dynamic libraries:

- Dynamic libraries (LIB, DLLs) can not be built by templates;
- future uses must be announced to force instances

```
template class Vector<int, 5>;
```

- And for methods:

```
template class Vector<int, 5>::Vector(int, int)
```

```
template int compara<int, int>(int, int)
```

CLASSES – TEMPLATE CLASSES

Specialization:

- define concrete situations in which the methods, functions, classes behave differently from the general situation
- have priority over the generic approach
- usually apply to some methods:

```
returned_type class_name<tipc>::method_name (params_list) { ... }
```

- you can specialize entire template classes:

```
template<> class_name<tipc> { ... }
```

CLASSES – TEMPLATE CLASSES

Extension:

```
template<typename T> class bt {...};
```

```
class b {...};
```

- template class derived from template class

```
template<typename T> class d: public bt<T> {...}
```

- template class derived from non template class

```
template<typename T> class d: public b {...}
```

- non template class derived from template class

```
template class d: public bt<int> {...}
```

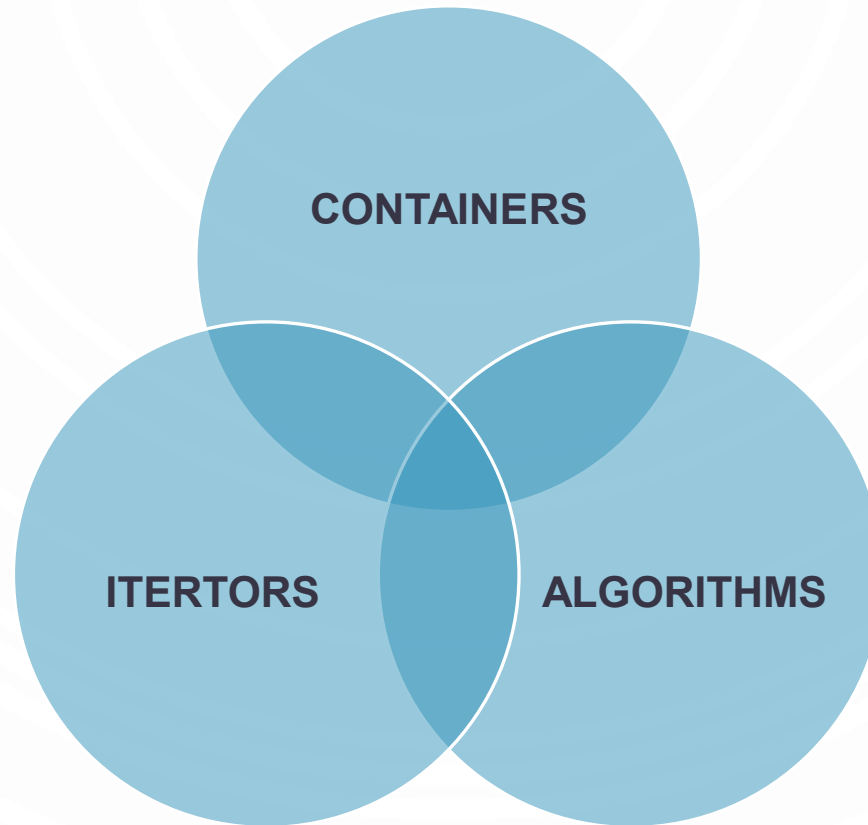
CLASSES – TEMPLATE CLASSES

- composition of template classes by inclusion
- composing template classes by parameterization with another template class

CLASSES – STL

- is a **Standard Template Library** (STL)
- covers the main data structures: vector, list, stack, queue, hash table;
- can be used without alterations for fundamental types or programmer defined types.

CLASSES – STL



CLASSES – STL (CONTAINERS)

- an object that stores other objects and has methods to access them;
- types (order and access function):
 - forward
 - reversible
 - random access
- Types (arrangement):
 - sequences
 - associative containers
 - container adaptors

CLASSES – STL (COLLECTIONS)

- sequential:
 - vector - dynamic contiguous array;
 - list - doubly-linked list ;
 - deque - double-ended queue;
- associative (value – key):
 - set - collection of unique keys, sorted by keys
 - multiset - collection of keys, sorted by keys
 - map - collection of key-value pairs, sorted by keys, keys are unique
 - multimap - collection of key-value pairs, sorted by keys
- adaptive: stack (LIFO) ; queue (FIFO) ; priority_queue

<http://en.cppreference.com/w/cpp/container>

CLASSES – STL



Salesman itinerary: List of locations (with duplicates)

List



Set

Salesman area: List of locations (NO duplicates)



Map

Salesman products managed by their name

CLASSES – STL (ITERATORS)

- generalized form of pointers;
- used to iterate through container elements
- interface between containers and algorithms
- predefined iterators:
 - `ostream_iterator`;
 - `istream_iterator`;
 - `reverse_iterator`;
 - `insert_iterator`;

CLASSES – STL ALGORITHMS

- generic functions independent of container type;
- used to process containers
- use iterators to access items
- important features:
 - copy;
 - for_each;
 - sort;
 - find;
 - transform

CLASSES – RTTI RUN-TIME TYPE IDENTIFICATION

- mechanism to determine at Run-Time the object type (managed by a base class pointer)
- makes sense in the context of a class hierarchy + upcasting + virtual functions

CLASSES – RTTI

- typeid() - Determines pointer destination type based on a type_info (typeinfo) structure

```
ComponenteGrafice::Model2D *pModel;  
  
pModel = new ComponenteGrafice::Dreptunghi();  
  
cout << typeid(*pModel).name() << endl;
```

CLASSES – RTTI

- RTTI - `dynamic_cast<T>()`
 - “type-safe downcast”;
 - este o functie template;
 - permite conversia la tipul T pentru un pointer la obiect de baza dca continutul de la adresa data de pointer este de tip T
 - evita erorile de conversie imposibile;
 - returneaza T* daca este posibil, altfe NULL;

CLASSES – RTTI

```
dynamic_cast<T>()
```

```
using namespace ComponenteGrafice;
```

```
pModel = new Dreptunghi();
```

```
if(dynamic_cast<Dreptunghi*>(pModel))
```

```
{
```

```
    cout<<endl<<"Continut de tip Dreptunghi !";
```

```
    Dreptunghi oDreptunghi = *dynamic_cast<Dreptunghi*>(pModel);
```

```
    oDreptunghi.Arie();
```

```
}
```